

CI/CD YAML syntax reference

Tier: Free, Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

This document lists the configuration options for the GitLab `.gitlab-ci.yml` file. This file is where you define the CI/CD jobs that make up your pipeline.

- If you are already familiar with [basic CI/CD concepts](#), try creating your own `.gitlab-ci.yml` file by following a tutorial that demonstrates a [simple](#) or [complex](#) pipeline.
- For a collection of examples, see [GitLab CI/CD examples](#).
- To view a large `.gitlab-ci.yml` file used in an enterprise, see the [.gitlab-ci.yml file for gitlab](#).

When you are editing your `.gitlab-ci.yml` file, you can validate it with the [CI Lint](#) tool.

GitLab CI/CD configuration uses YAML formatting, so the order of keywords is not important unless otherwise specified.

Use [CI/CD expressions](#) for more dynamic pipeline configuration options.

Keywords

A GitLab CI/CD pipeline configuration includes:

- [Global keywords](#) that configure pipeline behavior:

Keyword	Description
<code>default</code>	Custom default values for job keywords.
<code>include</code>	Import configuration from other YAML files.
<code>stages</code>	The names and order of the pipeline stages.
<code>variables</code>	Define default CI/CD variables for all jobs in the pipeline.
<code>workflow</code>	Control what types of pipeline run.

- [Header keywords](#)

Keyword	Description
<code>spec</code>	Define specifications for external configuration files.

- [Jobs](#) configured with [job keywords](#):

Keyword	Description
<code>after_script</code>	Override a set of commands that are executed after job.
<code>allow_failure</code>	Allow job to fail. A failed job does not cause the pipeline to fail.
<code>artifacts</code>	List of files and directories to attach to a job on success.
<code>before_script</code>	Override a set of commands that are executed before job.

<code>cache</code>	List of files that should be cached between subsequent runs.
<code>coverage</code>	Code coverage settings for a given job.
<code>dast_configuration</code>	Use configuration from DAST profiles on a job level.
<code>dependencies</code>	Restrict which artifacts are passed to a specific job by providing a list of jobs to fetch artifacts from.
<code>environment</code>	Name of an environment to which the job deploys.
<code>extends</code>	Configuration entries that this job inherits from.
<code>identity</code>	Authenticate with third party services using identity federation.
<code>image</code>	Use Docker images.
<code>inherit</code>	Select which global defaults all jobs inherit.
<code>interruptible</code>	Defines if a job can be canceled when made redundant by a newer run.
<code>manual_confirmation</code>	Define a custom confirmation message for a manual job.
<code>needs</code>	Execute jobs earlier than the stage ordering.
<code>pages</code>	Upload the result of a job to use with GitLab Pages.
<code>parallel</code>	How many instances of a job should be run in parallel.
<code>release</code>	Instructs the runner to generate a release object.
<code>resource_group</code>	Limit job concurrency.
<code>retry</code>	When and how many times a job can be auto-retried in case of a failure.
<code>rules</code>	List of conditions to evaluate and determine selected attributes of a job, and whether or not it's created.
<code>script</code>	Shell script that is executed by a runner.
<code>run</code>	Run configuration that is executed by a runner.
<code>secrets</code>	The CI/CD secrets the job needs.
<code>services</code>	Use Docker services images.
<code>stage</code>	Defines a job stage.
<code>start_in</code>	Delay job execution for a specified duration. Requires <code>when: delayed</code> .
<code>tags</code>	List of tags that are used to select a runner.
<code>timeout</code>	Define a custom job-level timeout that takes precedence over the project-wide setting.
<code>trigger</code>	Defines a downstream pipeline trigger.

<code>variables</code>	Define CI/CD variables for individual jobs.
<code>when</code>	When to run job.

- [Deprecated keywords](#) that are no longer recommended for use.

Global keywords

Some keywords are not defined in a job. These keywords control pipeline behavior or import additional pipeline configuration.

default

History >

You can set global defaults for some keywords. Each default keyword is copied to every job that doesn't already have it defined.

Default configuration does not merge with job configuration. If the job already has a keyword defined, the job keyword takes precedence and the default configuration for that keyword is not used.

Keyword type: Global keyword.

Supported values: These keywords can have custom defaults:

- `after_script`
- `artifacts`
- `before_script`
- `cache`
- `hooks`
- `id_tokens`
- `image`
- `interruptible`
- `retry`
- `services`
- `tags`

Example of `default`:

`yaml`

```
default:
  image: ruby:3.0
  retry: 2

rspec:
  script: bundle exec rspec

rspec 2.7:
  image: ruby:2.7
  script: bundle exec rspec
```

In this example:

- `image: ruby:3.0` and `retry: 2` are the default keywords for all jobs in the pipeline.
- The `rspec` job does not have `image` or `retry` defined, so it uses the defaults of `image: ruby:3.0` and `retry: 2`.

Additional details:

- Control inheritance of default keywords in jobs with `inherit:default`.
- Global defaults are not passed to [downstream pipelines](#), which run independently of the upstream pipeline that triggered the downstream pipeline.

include

Use `include` to include external YAML files in your CI/CD configuration. You can split one long `.gitlab-ci.yml` file into multiple files to increase readability, or reduce duplication of the same configuration in multiple places.

You can also store template files in a central repository and include them in projects.

The `include` files are:

- Merged with those in the `.gitlab-ci.yml` file.
- Always evaluated first and then merged with the content of the `.gitlab-ci.yml` file, regardless of the position of the `include` keyword.

The time limit to resolve all files is 30 seconds.

Keyword type: Global keyword.

Supported values: The `include` subkeys:

- `include:component`
- `include:local`
- `include:project`
- `include:remote`
- `include:template`

And optionally:

- `include:inputs`
- `include:rules`
- `include:integrity`
- `include:cache`

Additional details:

- Only [certain CI/CD variables](#) can be used with `include` keywords.
- Use merging to customize and override included CI/CD configurations with local
- You can override included configuration by having the same job name or global keyword in the `.gitlab-ci.yml` file. The two configurations are merged together, and the configuration in the `.gitlab-ci.yml` file takes precedence over the included configuration.
- If you rerun a:
 - Job, the `include` files are not fetched again. All jobs in a pipeline use the configuration fetched when the pipeline was created. Any changes to the source `include` files do not affect job reruns.
 - Pipeline, the `include` files are fetched again. If they changed after the last pipeline run, the new pipeline uses the changed configuration.
- You can have up to 150 includes per pipeline by default, including [nested](#). Additionally:
 - In [GitLab 16.0 and later](#) users on GitLab Self-Managed can change the [maximum includes](#) value.
 - In [GitLab 15.10 and later](#) you can have up to 150 includes. In nested includes, the same file can be included multiple times, but duplicated includes count towards the limit.

include:component

Use `include:component` to add a [CI/CD component](#) to the pipeline configuration.

Keyword type: Global keyword.

Supported values: The full address of the CI/CD component, formatted as `<fully-qualified-domain-name>/<project-path>/<component-name>@<specific-version>`.

Example of `include:component`:

yaml


```
include:
  - component: $CI_SERVER_FQDN/my-org/security-components/secret-detection@1.0
```

Related topics:

- [Use a CI/CD component.](#)

include:local

Use `include:local` to include a file that is in the same repository and branch as the configuration file containing the `include` keyword. Use `include:local` instead of symbolic links.

Keyword type: Global keyword.

Supported values:

A full path relative to the root directory (/):

- The YAML file must have the extension `.yaml` or `.yml`.
- You can [use](#) `*` and `**` [wildcards in the file path](#).
- You can use [certain CI/CD variables](#).

Example of `include:local`:

yaml


```
include:
  - local: '/templates/.gitlab-ci-template.yml'
```

You can also use shorter syntax to define the path:

yaml


```
include: '.gitlab-ci-production.yml'
```

Additional details:

- The `.gitlab-ci.yml` file and the local file must be on the same branch.
- You can't include local files through Git submodules paths.
- `include` configuration is always evaluated based on the location of the file containing the `include` keyword, not the project running the pipeline. If a [nested include](#) is in a configuration file in a different project, `include: local` checks that other project for the file.

Keyword type: Global keyword.

Supported values:

- `include:project`: The full GitLab project path.
- `include:file` A full file path, or array of file paths, relative to the root directory (`/`). The YAML files must have the `.yaml` or `.yml` extension.
- `include:ref`: Optional. The ref to retrieve the file from. Defaults to the `HEAD` of the project when not specified.
- You can use [certain CI/CD variables](#).

Example of `include:project`:

yaml

```
include:
  - project: 'my-group/my-project'
    file: '/templates/.gitlab-ci-template.yaml'
  - project: 'my-group/my-subgroup/my-project-2'
    file:
      - '/templates/.builds.yaml'
      - '/templates/.tests.yaml'
```

You can also specify a `ref`:

yaml

```
include:
  - project: 'my-group/my-project'
    ref: main # Git branch
    file: '/templates/.gitlab-ci-template.yaml'
  - project: 'my-group/my-project'
    ref: v1.0.0 # Git Tag
    file: '/templates/.gitlab-ci-template.yaml'
  - project: 'my-group/my-project'
    ref: 787123b47f14b552955ca2786bc9542ae66fee5b # Git SHA
    file: '/templates/.gitlab-ci-template.yaml'
```

Additional details:

- `include` configuration is always evaluated based on the location of the file containing the `include` keyword, not the project running the pipeline. If a [nested include](#) is in a configuration file in a different project, `include: local` checks that other project for the file.
- When the pipeline starts, the `.gitlab-ci.yml` file configuration included by all methods is evaluated. The configuration is a snapshot in time and persists in the database. GitLab does not reflect any changes to the referenced `.gitlab-ci.yml` file configuration until the next pipeline starts.
- When you include a YAML file from another private project, the user running the pipeline must be a member of both projects and have the appropriate permissions to run pipelines. A `not found or access denied` error may be displayed if the user does not have access to any of the included files.
- Be careful when including another project's CI/CD configuration file. No pipelines or notifications trigger when CI/CD configuration files change. From a security perspective, this is similar to pulling a third-party dependency. For the `ref`, consider:
 - Using a specific SHA hash, which should be the most stable option. Use the full 40-character SHA hash to ensure the desired commit is referenced, because using a short SHA hash for the `ref` might be ambiguous.
 - Applying both [protected branch](#) and [protected tag](#) rules to the `ref` in the other project. Protected tags and branches are more likely to pass through change management before changing.

Keyword type: Global keyword.

Supported values:

A public URL accessible by an HTTP/HTTPS `GET` request:

- Authentication with the remote URL is not supported.
- The YAML file must have the extension `.yaml` or `.yml`.
- You can use [certain CI/CD variables](#).

Example of `include:remote:`

yaml


```
include:
  - remote: 'https://gitlab.com/example-project/-/raw/main/.gitlab-ci.yml'
```

Additional details:

- All [nested includes](#) are executed without context as a public user, so you can only include public projects or templates. No variables are available in the `include` section of nested includes.
- Be careful when including another project's CI/CD configuration file. No pipelines or notifications trigger when the other project's files change. From a security perspective, this is similar to pulling a third-party dependency. To verify the integrity of the included file, consider using the `integrity` keyword. If you link to another GitLab project you own, consider the use of both [protected branches](#) and [protected tags](#) to enforce change management rules.

`include:template`

Use `include:template` to include [.gitlab-ci.yml templates](#).

Keyword type: Global keyword.

Supported values:

- The filename of a CI/CD template, for example `Auto-DevOps.gitlab-ci.yml`.
- You can use [certain CI/CD variables](#).

Example of `include:template:`

yaml


```
# File sourced from the GitLab template collection
include:
  - template: Auto-DevOps.gitlab-ci.yml
```

Multiple `include:template` files:

yaml


```
include:
  - template: Android-Fastlane.gitlab-ci.yml
  - template: Auto-DevOps.gitlab-ci.yml
```

Additional details:

- All templates can be viewed in [lib/gitlab/ci/templates](#). Not all templates are designed to be used with `include:template`, so check template comments before using one.

include:inputs

History >

Use `include:inputs` to set the values for input parameters when the included configuration uses `spec:inputs` and is added to the pipeline.

Keyword type: Global keyword.

Supported values: A string, numeric value, or boolean.

Example of `include:inputs`:

yaml

```
include:
  - local: 'custom_configuration.yml'
    inputs:
      website: "My website"
```

In this example:

- The configuration contained in `custom_configuration.yml` is added to the pipeline, with a `website` input set to a value of `My website` for the included configuration.

Additional details:

- If the included configuration file uses `spec:inputs:type`, the input value must match the defined type.
- If the included configuration file uses `spec:inputs:options`, the input value must match one of the listed options.

Related topics:

- [Set input values when using `include`](#).

include:rules

You can use `rules` with `include` to conditionally include other configuration files.

Keyword type: Global keyword.

Supported values: These `rules` subkeys:

- `rules:if`.
- `rules:exists`.
- `rules:changes`.

Some [CI/CD variables are supported](#).

Example of `include:rules`:

yaml


```
- if: $INCLUDE_BUILDS == "true"

test-job:
  stage: test
  script: echo "This is a test job"
```

In this example, if the `INCLUDE_BUILDS` variable is:

- `true`, the `build_jobs.yml` configuration is included in the pipeline.
- Not `true` or does not exist, the `build_jobs.yml` configuration is not included in the pipeline.

Related topics:

- Examples of using `include` with:
 - `rules:if`.
 - `rules:changes`.
 - `rules:exists`.

`include:integrity`

History >

Use `integrity` with `include:remote` to specify a SHA256 hash of the included remote file. If `integrity` does not match the actual content, the remote file is not processed and the pipeline fails.

Keyword type: Global keyword.

Supported values: Base64-encoded SHA256 hash of the included content.

Example of `include:integrity`:

yaml

```
include:
  - remote: 'https://gitlab.com/example-project/-/raw/main/.gitlab-ci.yml'
    integrity: 'sha256-L3/GAoKaw0Arw6hDCKeKQLV1QPEgHYxGBHsH4zG1IY8='
```

`include:cache`

Status: Experiment

History >

 The availability of this feature is controlled by a feature flag. For more information, see the history. This feature is available for testing, but not ready for production use.

Use `cache` with `include:remote` to cache the fetched remote file content and reduce HTTP requests. When enabled, the remote file is cached for a specified time-to-live (TTL), improving pipeline performance for configurations that use the same remote includes repeatedly.

Consider the trade-off between performance and freshness when setting cache durations. Longer cache durations improve performance but might use stale content if the remote file changes frequently.

When `cache` is not defined, the remote file is fetched every time.

- `true`: Enable caching with a default time-to-live (TTL) of 1 hour.
- A duration (string): Valid TTL duration strings use time units like `minutes`, `hours`, or `days` (minimum `1 minute`).

Example of `include:cache`:

yaml

```
include:
  - remote: 'https://gitlab.com/example-project/-/raw/main/sample1.gitlab-ci.yml'
    cache: true
  - remote: 'https://gitlab.com/example-project/-/raw/main/sample2.gitlab-ci.yml'
    cache: '1 day'
```

Additional details:

- Caching is only available for `include:remote`.
- After the remote file is cached, the cached version continues to be used until the TTL expires, even if the remote file content changes.
- If you use `integrity` with `cache`, the integrity check is performed on every pipeline run, even when using cached content.

stages

History >

Use `stages` to define stages that contain groups of jobs. Use `stage` in a job to configure the job to run in a specific stage.

If `stages` is not defined in the `.gitlab-ci.yml` file, the default pipeline stages are:

- `.pre`
- `build`
- `test`
- `deploy`
- `.post`

The order of the items in `stages` defines the execution order for jobs:

- Jobs in the same stage run in parallel.
- Jobs in the next stage run after the jobs from the previous stage complete successfully.

If a pipeline contains only jobs in the `.pre` or `.post` stages, it does not run. There must be at least one other job in a different stage.

Keyword type: Global keyword.

Example of `stages`:

yaml

```
stages:
  - build
  - test
  - deploy
```

In this example:

1. All jobs in `build` execute in parallel.
2. If all jobs in `build` succeed, the `test` jobs execute in parallel.

If any job fails, the pipeline is marked as `failed` and jobs in later stages do not start. Jobs in the current stage are not stopped and continue to run.

Additional details:

- If a job does not specify a `stage`, the job is assigned the `test` stage.
- If a stage is defined but no jobs use it, the stage is not visible in the pipeline, which can help [compliance pipeline configurations](#):
 - Stages can be defined in the compliance configuration but remain hidden if not used.
 - The defined stages become visible when developers use them in job definitions.

Related topics:

- To make a job start earlier and ignore the stage order, use the `needs` keyword.

workflow

Use `workflow` to control pipeline behavior.

You can use some [predefined CI/CD variables](#) in `workflow` configuration, but not variables that are only defined when jobs start.

Related topics:

- [workflow: rules examples](#)
- [Switch between branch pipelines and merge request pipelines](#)

workflow:auto_cancel:on_new_commit

History >

Use `workflow:auto_cancel:on_new_commit` to configure the behavior of the [auto-cancel redundant pipelines](#) feature.

Supported values:

- `conservative`: Cancel the pipeline, but only if no jobs with `interruptible: false` have started yet. Default when not defined.
- `interruptible`: Cancel only jobs with `interruptible: true`.
- `none`: Do not auto-cancel any jobs.

Example of `workflow:auto_cancel:on_new_commit`:

yaml

```
workflow:
  auto_cancel:
    on_new_commit: interruptible

job1:
  interruptible: true
  script: sleep 60

job2:
  interruptible: false # Default when not defined.
  script: sleep 60
```

In this example:

- When a new commit is pushed to a branch, GitLab creates a new pipeline and `job1` and `job2` start.

workflow:auto_cancel:on_job_failure

History >

Use workflow:auto_cancel:on_job_failure to configure which jobs should be canceled as soon as one job fails.

Supported values:

- all: Cancel the pipeline and all running jobs as soon as one job fails.
- none: Do not auto-cancel any jobs.

Example of workflow:auto_cancel:on_job_failure:

yaml

```
stages: [stage_a, stage_b]

workflow:
  auto_cancel:
    on_job_failure: all

job1:
  stage: stage_a
  script: sleep 60

job2:
  stage: stage_a
  script:
    - sleep 30
    - exit 1

job3:
  stage: stage_b
  script:
    - sleep 30
```

In this example, if job2 fails, job1 is canceled if it is still running and job3 does not start.

Related topics:

- [Auto-cancel the parent pipeline from a downstream pipeline](#)

workflow:name

History >

You can use name in workflow: to define a name for pipelines.

All pipelines are assigned the defined name. Any leading or trailing spaces in the name are removed.

Supported values:

- A string.
- [CI/CD variables](#).
- A combination of both.

Examples of workflow:name:

A simple pipeline name with a predefined variable:

```
name: 'Pipeline for branch: $CI_COMMIT_BRANCH'
```

A configuration with different pipeline names depending on the pipeline conditions:

yaml



```
variables:
  PROJECT1_PIPELINE_NAME: 'Default pipeline name' # A default is not required

workflow:
  name: '$PROJECT1_PIPELINE_NAME'
  rules:
    - if: '$CI_MERGE_REQUEST_LABELS =~ /pipeline:run-in-ruby3/'
      variables:
        PROJECT1_PIPELINE_NAME: 'Ruby 3 pipeline'
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      variables:
        PROJECT1_PIPELINE_NAME: 'MR pipeline: $CI_MERGE_REQUEST_SOURCE_BRANCH_NAME'
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH # For default branch pipelines, use the default name
```

Additional details:

- If the name is an empty string, the pipeline is not assigned a name. A name consisting of only CI/CD variables could evaluate to an empty string if all the variables are also empty.
- `workflow:rules:variables` become [default variables](#) available in all jobs, including [trigger](#) jobs which forward variables to downstream pipelines by default. If the downstream pipeline uses the same variable, the [variable is overwritten](#) by the upstream variable value. Be sure to either:
 - Use a unique variable name in every project’s pipeline configuration, like `PROJECT1_PIPELINE_NAME`.
 - Use [inherit:variables](#) in the trigger job and list the exact variables you want to forward to the downstream pipeline.

workflow:rules

The `rules` keyword in `workflow` is similar to [rules defined in jobs](#), but controls whether or not a whole pipeline is created.

When no rules evaluate to true, the pipeline does not run.

Supported values: You can use some of the same keywords as job-level [rules](#):

- `rules: if`.
- `rules: changes`.
- `rules: exists`.
- `when`, can only be `always` or `never` when used with `workflow`.
- `variables`.

Example of `workflow:rules`:

yaml



```
workflow:
  rules:
    - if: $CI_COMMIT_TITLE =~ /-draft$/
      when: never
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

In this example, pipelines run if the commit title (first line of the commit message) does not end with `-draft` and the pipeline is for either:

Additional details:

- If your rules match both branch pipelines (other than the default branch) and merge request pipelines, [duplicate pipelines](#) can occur.
- `start_in`, `allow_failure`, and `needs` are not supported in `workflow:rules`, but do not cause a syntax violation. Though they have no effect, do not use them in `workflow:rules` as it could cause syntax failures in the future. See [issue 436473](#) for more details.

Related topics:

- [Common `if` clauses for `workflow:rules`](#).
- [Use `rules` to run merge request pipelines](#).

workflow:rules:variables

You can use [variables](#) in `workflow:rules` to define variables for specific pipeline conditions.

When the condition matches, the variable is created and can be used by all jobs in the pipeline. If the variable is already defined at the top level as a default variable, the `workflow` variable takes precedence and overrides the default variable.

Keyword type: Global keyword.

Supported values: Variable name and value pairs:

- The name can use only numbers, letters, and underscores (`_`).
- The value must be a string.

Example of `workflow:rules:variables`:

yaml

```
variables:
  DEPLOY_VARIABLE: "default-deploy"

workflow:
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      variables:
        DEPLOY_VARIABLE: "deploy-production" # Override globally-defined DEPLOY_VARIABLE
    - if: $CI_COMMIT_BRANCH =~ /feature/
      variables:
        IS_A_FEATURE: "true" # Define a new variable.
    - if: $CI_COMMIT_BRANCH # Run the pipeline in other cases

job1:
  variables:
    DEPLOY_VARIABLE: "job1-default-deploy"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      variables:
        DEPLOY_VARIABLE: "job1-deploy-production" # Override DEPLOY_VARIABLE defined
        DEPLOY_VARIABLE: "job1-deploy-production" # at the job level.
    - when: on_success # Run the job in other cases
  script:
    - echo "Run script with $DEPLOY_VARIABLE as an argument"
    - echo "Run another script if $IS_A_FEATURE exists"

job2:
  script:
    - echo "Run script with $DEPLOY_VARIABLE as an argument"
    - echo "Run another script if $IS_A_FEATURE exists"
```


- job2's `DEPLOY_VARIABLE` is `deploy-production`.

When the branch is `feature`:

- job1's `DEPLOY_VARIABLE` is `job1-default-deploy`, and `IS_A_FEATURE` is `true`.
- job2's `DEPLOY_VARIABLE` is `default-deploy`, and `IS_A_FEATURE` is `true`.

When the branch is something else:

- job1's `DEPLOY_VARIABLE` is `job1-default-deploy`.
- job2's `DEPLOY_VARIABLE` is `default-deploy`.

Additional details:

- `workflow:rules:variables` become [default variables](#) available in all jobs, including [trigger](#) jobs which forward variables to downstream pipelines by default. If the downstream pipeline uses the same variable, the [variable is overwritten](#) by the upstream variable value. Be sure to either:
 - Use unique variable names in every project's pipeline configuration, like `PROJECT1_VARIABLE_NAME`.
 - Use [inherit:variables](#) in the trigger job and list the exact variables you want to forward to the downstream pipeline.

`workflow:rules:auto_cancel`

History >

Use `workflow:rules:auto_cancel` to configure the behavior of the [workflow:auto_cancel:on_new_commit](#) or the [workflow:auto_cancel:on_job_failure](#) features.

Supported values:

- `on_new_commit`: [workflow:auto_cancel:on_new_commit](#)
- `on_job_failure`: [workflow:auto_cancel:on_job_failure](#)

Example of `workflow:rules:auto_cancel`:

yaml

```
workflow:
  auto_cancel:
    on_new_commit: interruptible
    on_job_failure: all
  rules:
    - if: $CI_COMMIT_REF_PROTECTED == 'true'
      auto_cancel:
        on_new_commit: none
        on_job_failure: none
    - when: always                # Run the pipeline in other cases

test-job1:
  script: sleep 10
  interruptible: false

test-job2:
  script: sleep 10
  interruptible: true
```

In this example, [workflow:auto_cancel:on_new_commit](#) is set to `interruptible` and [workflow:auto_cancel:on_job_failure](#) is set to `all` for all jobs by default. But if a pipeline runs for a protected branch, the rule overrides the default with `on_new_commit: none` and `on_job_failure: none`. For example, if a pipeline is running for:

- A non-protected branch and a new commit is pushed, `test-job1` continues to run and `test-job2` is canceled.

Header keywords

Some keywords must be defined in a header section of a YAML configuration file. The header must be at the top of the file, separated from the rest of the configuration with `---`.

spec

History >

Add a `spec` section to the header of a YAML file to configure the behavior of a pipeline when a configuration is added to the pipeline with the `include` keyword.

Specs must be declared at the top of a configuration file, in a header section separated from the rest of the configuration with `--`.

spec:inputs

You can use `spec:inputs` to define [inputs](#) for the CI/CD configuration.

Use the interpolation format `${[ inputs.input-id ]}` to reference the values outside of the header section. Inputs are evaluated and interpolated when the configuration is fetched during pipeline creation. When using `inputs`, interpolation completes before the configuration is merged with the contents of the `.gitlab-ci.yml` file.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: A hash of strings representing the expected inputs.

Example of `spec:inputs`:

yaml

```
spec:
  inputs:
    environment:
    job-stage:
  ---

scan-website:
  stage: ${[ inputs.job-stage ]}
  script: ./scan-website ${[ inputs.environment ]}
```

Additional details:

- Inputs are mandatory unless you use `spec:inputs:default` to set a default value. Avoid mandatory inputs unless you only use inputs with `include:inputs`.
- Inputs expect strings unless you use `spec:inputs:type` to set a different input type.
- A string containing an interpolation block must not exceed 1 MB.
- The string inside an interpolation block must not exceed 1 KB.
- You can define input values [when running a new pipeline](#).

Related topics:

- [Define input parameters with `spec:inputs`](#).

spec:inputs:default

Inputs are mandatory when included, unless you set a default value with `spec:inputs:default`.

Use `default: ''` to have no default value.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: A string representing the default value, or `''`.

Example of `spec:inputs:default`:

yaml

```
spec:
  inputs:
    website:
    user:
      default: 'test-user'
    flags:
      default: ''
---
# The pipeline configuration would follow...
```

In this example:

- `website` is mandatory and must be defined.
- `user` is optional. If not defined, the value is `test-user`.
- `flags` is optional. If not defined, it has no value.

Additional details:

- The pipeline fails with a validation error when the input:
 - Uses both `default` and `options`, but the default value is not one of the listed options.
 - Uses both `default` and `regex`, but the default value does not match the regular expression.
 - Value does not match the `type`.

spec:inputs:description

History >

Use `description` to give a description to a specific input. The description does not affect the behavior of the input and is only used to help users of the file understand the input.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: A string representing the description.

Example of `spec:inputs:description`:

yaml

```
spec:
  inputs:
    flags:
      description: 'Sample description of the `flags` input details.'
---
# The pipeline configuration would follow...
```

Inputs can use `options` to specify a list of allowed values for an input. The limit is 50 options per input.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: An array of input options.

Example of `spec:inputs:options`:

yaml

```
spec:
  inputs:
    environment:
      options:
        - development
        - staging
        - production
---
# The pipeline configuration would follow...
```

In this example:

- `environment` is mandatory and must be defined with one of the values in the list.

Additional details:

- The pipeline fails with a validation error when:
 - The input uses both `options` and `default`, but the default value is not one of the listed options.
 - Any of the input options do not match the `type`, which can be either `string` or `number`, but not `boolean` when using `options`.

spec:inputs:regex

History >

Use `spec:inputs:regex` to specify a regular expression that the input must match.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: Must be a regular expression.

Example of `spec:inputs:regex`:

yaml

```
spec:
  inputs:
    version:
      regex: ^v\d\.\d+(\.\d+)?$
---
# The pipeline configuration would follow...
```

In this example, inputs of `v1.0` or `v1.2.3` match the regular expression and pass validation. An input of `v1.A.B` does not match the regular expression and fails validation.

Additional details:

- `inputs:regex` can only be used with a `type` of `string`, not `number` or `boolean`.

- Validation of the input against the regular expression happens before variable expansion. If the input text includes a variable name, the raw value of the input (the variable name) is validated, not the variable value.

spec:inputs:rules

History >

Use `spec:inputs:rules` to define conditional `options` and `default` values for an input based on the values of other inputs.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: An array of rule objects. Each rule can have:

- `if`: A conditional expression to check input values, using `$$[ inputs.input-id ]` [syntax](#).
- `options`: An array of allowed values for the input.
- `default`: The default value for the input when this rule matches. Use `default: null` to allow users to enter their own value for the input.

Example of `spec:inputs:rules`:

yaml

```
spec:
  inputs:
    environment:
      options: ['development', 'production']
      default: 'development'

    instance_type:
      description: 'VM instance size'
      rules:
        - if: $$[ inputs.environment ] == 'development'
          options: ['small', 'medium']
          default: 'small'
        - if: $$[ inputs.environment ] == 'production'
          options: ['large', 'xlarge']
          default: 'large'
  ---

  deploy:
    script: echo "Deploying $$[ inputs.instance_type ] instance"
```

In this example, when `environment` is `development`, users can only select `small` or `medium` instances. When `environment` is `production`, only `large` or `xlarge` instances are available.

Additional details:

- Rules are evaluated in order. The first rule with a matching `if` condition is used.
- A rule without an `if` condition acts as a fallback when no other rules match.
- Fallback rules must define `options` with at least one value.
- All rules with `options` must also define a `default` value that exists in the `options` list.
- You cannot use both `rules` and top-level `options` or `default` for the same input.

Related topics:

- [Define conditional input options with spec:inputs:rules](#).

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: Can be one of:

- `array`, to accept an [array](#) of inputs.
- `string`, to accept string inputs (default when not defined).
- `number`, to only accept numeric inputs.
- `boolean`, to only accept `true` or `false` inputs.

Example of `spec:inputs:type`:

yaml

```

spec:
  inputs:
    job_name:
    website:
      type: string
    port:
      type: number
    available:
      type: boolean
    array_input:
      type: array
---
# The pipeline configuration would follow...
```

spec:include

History >

Use `spec:include` to include external input definitions from other files. You can share and reuse input definitions across multiple pipeline configurations.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: An array of include locations. Supports `local`, `remote`, and `project` includes only.

Example of `spec:include`:

yaml

```

spec:
  include:
    - local: /shared-inputs.yml
  inputs:
    environment:
      default: production
---

deploy:
  script: echo "Deploying to $[[ inputs.environment ]]"
```

With multiple includes from different sources:

yaml

```
- remote: 'https://example.com/ci/common-inputs.yml'
- project: 'my-group/shared-configs'
  ref: main
  file: '/ci/team-inputs.yml'
inputs:
  environment:
    default: production
---

deploy:
  script: echo "Deploying to $[ inputs.environment ]"
```

Additional details:

- You cannot use `spec:include` in [CI/CD components](#).
- External input files must contain only the `inputs` key. Other keys cause validation errors.
- External inputs are merged first, then inline inputs are applied.
- Inline inputs cannot have the same name as included inputs.
- When you include multiple input files, they are merged in the order specified.
- Supports `local`, `remote`, and `project` include types. Does not support `template`, `component`, or `artifact` includes.

Related topics:

- [Use inputs from external files](#).

`spec:component`

History >

Use `spec:component` to define which component context data is available for interpolation in a [CI/CD component](#).

Component context provides metadata about the component itself, such as its name, version, and the commit SHA. This allows component templates to reference their own metadata dynamically.

Use the interpolation format `$[ component.field-name ]` to reference component context values in the component template.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: An array of strings. Each string must be one of:

- `name`: The component name as specified in the component path.
- `sha`: The commit SHA of the component.
- `version`: The resolved semantic version from the catalog resource. Returns `null` if:
 - The component is not a catalog resource.
 - The reference is a branch name or commit SHA (not a released version).
- `reference`: The original reference specified after `@` in the component path. For example, `1.0`, `~latest`, a branch name, or a commit SHA.

Example of `spec:component`:

yaml




```
  stage:
    default: build
  ---

build-image:
  stage: $[[ inputs.stage ]]
  image: registry.example.com/$[[ component.name ]]:$[[ component.version ]]
  script:
    - echo "Building with component version $[[ component.version ]]"
    - echo "Component reference: $[[ component.reference ]]"
```

Additional details:

- The `version` field resolves to the actual semantic version when using:
 - A full version like `@1.0.0` (returns `1.0.0`)
 - A partial version like `@1.0` (returns the latest matching version, for example `1.0.2`)
 - `@~latest` (returns the latest version)
- The `reference` field always returns the exact value specified after `@`:
 - `@1.0` returns `1.0` (while `version` might return `1.0.2`)
 - `@~latest` returns `~latest` (while `version` returns the actual version number)
 - `@abc123` returns `abc123` (while `version` returns `null`)

Related topics:

- [Use component context in components.](#)

`spec:description`

History >

Use `spec:description` to provide a short description of the component. The description is displayed in the CI/CD Catalog on the component details page, above the inputs table.

Keyword type: Header keyword. `spec` must be declared at the top of the configuration file, in a header section.

Supported values: A string describing the component.

Example of `spec:description`:

yaml

```
spec:
  description: "A description of the component visible to users in the CI/CD Catalog."
  inputs:
    stage:
      default: test
  ---

scan-job:
  stage: $[[ inputs.stage ]]
  script: ./run-scan.sh
```

Job keywords

The following topics explain how to use keywords to configure CI/CD pipelines.

Use `after_script` to define an array of commands to run last, after a job's `before_script` and `script` sections complete. `after_script` commands also run when:

- The job is canceled while the `before_script` or `script` sections are still running.
- The job fails with failure type of `script_failure`, but not [other failure types](#).

Job configuration and default configuration does not merge together. If the pipeline has `default:after_script` defined, and the job also has `after_script`, the job configuration takes precedence and the default configuration is not used.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` [section](#).

Supported values: An array including:

- Single line commands.
- Long commands [split over multiple lines](#).
- [YAML anchors](#).

CI/CD variables [are supported](#).

Example of `after_script`:

yaml


```

job:
  script:
    - echo "An example script section."
  after_script:
    - echo "Execute this command after the `script` section completes."

```

Additional details:

Scripts you specify in `after_script` execute in a new shell, separate from any `before_script` or `script` commands. As a result, they:

- Have the current working directory set back to the default (according to the [variables which define how the runner processes Git requests](#)).
- Don't have access to changes done by commands defined in the `before_script` or `script`, including:
 - Command aliases and variables exported in `script` scripts.
 - Changes outside of the working tree (depending on the runner executor), like software installed by a `before_script` or `script` script.
- Have a separate timeout. For GitLab Runner 16.4 and later, this defaults to 5 minutes, and can be configured with the `RUNNER_AFTER_SCRIPT_TIMEOUT` variable. In GitLab 16.3 and earlier, the timeout is hard-coded to 5 minutes.
- Don't affect the job's exit code. If the `script` section succeeds and the `after_script` times out or fails, the job exits with code `0` (Job Succeeded).
- For jobs that time out:
 - `after_script` commands do not execute by default.
 - You can [configure timeout values](#) to ensure `after_script` runs by setting appropriate `RUNNER_SCRIPT_TIMEOUT` and `RUNNER_AFTER_SCRIPT_TIMEOUT` values that don't exceed the job's timeout.
- Using `after_script` at the top level, but not in the `default` section, is [deprecated](#).

Execution timing and file inclusion:

`after_script` commands execute before cache and artifact upload operations.

- If you configured artifact collection:
 - Files created or modified in `after_script` are included in artifacts.

use this timing for scenarios like:

- Generating test reports or coverage data after the main script.
- Creating summary files or logs.
- Post-processing build outputs.

In the following example, the only files that are not included are those created or modified after the artifact or cache upload stages:

yaml


```

job:
  script:
    - echo "main" > output.txt
    - build_something

  after_script:
    - echo "modified in after_script" >> output.txt  # This WILL be in the artifact
    - generate_test_report > report.html             # This WILL be in the artifact

  artifacts:
    paths:
      - output.txt
      - report.html

  cache:
    paths:
      - output.txt  # Will include the "modified in after_script" line
  
```

For more information, see [job execution flow](#).

Related topics:

- [Use `after\_script` with `default`](#) to define a default array of commands that should run after all jobs.
- You can configure a job to [skip `after\_script` commands if the job is canceled](#).
- You can [ignore non-zero exit codes](#).
- [Use color codes with `after\_script`](#) to make job logs easier to review.
- [Create custom collapsible sections](#) to simplify job log output.
- You can [ignore errors in `after\_script`](#).

allow_failure

Use `allow_failure` to determine whether a pipeline should continue running when a job fails.

- To let the pipeline continue running subsequent jobs, use `allow_failure: true`.
- To stop the pipeline from running subsequent jobs, use `allow_failure: false`.

When jobs are allowed to fail (`allow_failure: true`) an orange warning () indicates that a job failed. However, the pipeline is successful and the associated commit is marked as passed with no warnings.

This same warning is displayed when:

- All other jobs in the stage are successful.
- All other jobs in the pipeline are successful.

The default value for `allow_failure` is:

- `true` for [manual jobs](#).
- `false` for jobs that use `when: manual` inside [rules](#).

Supported values:

- `true` or `false`.

Example of `allow_failure`:

yaml

```
job1:
  stage: test
  script:
    - execute_script_1

job2:
  stage: test
  script:
    - execute_script_2
  allow_failure: true

job3:
  stage: deploy
  script:
    - deploy_to_staging
  environment: staging
```

In this example, `job1` and `job2` run in parallel:

- If `job1` fails, jobs in the `deploy` stage do not start.
- If `job2` fails, jobs in the `deploy` stage can still start.

Additional details:

- You can use `allow_failure` as a subkey of `rules`.
- If `allow_failure: true` is set, the job is always considered successful, and later jobs with `when: on_failure` don't start if this job fails.
- You can use `allow_failure: false` with a manual job to create a [blocking manual job](#). A blocked pipeline does not run any jobs in later stages until the manual job is started and completes successfully.

`allow_failure:exit_codes`

Use `allow_failure:exit_codes` to control when a job should be allowed to fail. The job is `allow_failure: true` for any of the listed exit codes, and `allow_failure` false for any other exit code.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- A single exit code.
- An array of exit codes.

Example of `allow_failure`:

yaml

```
- exit 1
allow_failure:
  exit_codes: 137

test_job_2:
  script:
    - echo "Run a script that results in exit code 137. This job is allowed to fail."
    - exit 137
  allow_failure:
    exit_codes:
      - 137
      - 255
```

artifacts

History >

Use `artifacts` to specify which files to save as [job artifacts](#). Job artifacts are a list of files and directories that are attached to the job when it [succeeds, fails, or always](#).

The artifacts are sent to GitLab after the job finishes. They are available for download in the GitLab UI if the size is smaller than the [maximum artifact size](#).

By default, jobs in later stages automatically download all the artifacts created by jobs in earlier stages. You can control artifact download behavior in jobs with [dependencies](#).

When using the `needs` keyword, jobs can only download artifacts from the jobs defined in the `needs` configuration.

Job artifacts are only collected for successful jobs by default, and artifacts are restored after [caches](#).

Job configuration and default configuration does not merge together. If the pipeline has `default:artifacts` defined, and the job also has `artifacts`, the job configuration takes precedence and the default configuration is not used.

[Read more about artifacts](#).

artifacts:paths

Paths are relative to the project directory (`$CI_PROJECT_DIR`) and can't directly link outside it.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- An array of file paths, relative to the project directory.
- You can use Wildcards that use [glob](#) [↗](#) patterns and [doublestar.Glob](#) [↗](#) patterns.
- For [GitLab Pages job](#):
 - In [GitLab 17.10 and later](#), the `pages.publish` path is automatically appended to `artifacts:paths`, so you don't need to specify it again.
 - In [GitLab 17.10 and later](#), when the `pages.publish` path is not specified, the `public` directory is automatically appended to `artifacts:paths`.

CI/CD variables [are supported](#).

Example of `artifacts:paths`:

yaml



- binaries/
- .config

This example creates an artifact with `.config` and all the files in the `binaries` directory.

Additional details:

- If not used with `artifacts:name`, the artifacts file is named `artifacts`, which becomes `artifacts.zip` when downloaded.

Related topics:

- To restrict which jobs a specific job fetches artifacts from, see [dependencies](#).
- [Create job artifacts](#).

artifacts:exclude

Use `artifacts:exclude` to prevent files from being added to an artifacts archive.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- An array of file paths, relative to the project directory.
- You can use Wildcards that use [glob](#) or [doublestar.PathMatch](#) patterns.

Example of artifacts:exclude:

yaml

```
artifacts:
  paths:
    - binaries/
  exclude:
    - binaries/**/*.*
```

This example stores all files in `binaries/`, but not `*.*` files located in subdirectories of `binaries/`.

Additional details:

- `artifacts:exclude` paths are not searched recursively.
- Files matched by [artifacts:untracked](#) can be excluded using `artifacts:exclude` too.

Related topics:

- [Exclude files from job artifacts](#).

artifacts:expire_in

Use `expire_in` to specify how long [job artifacts](#) are stored before they expire and are deleted. The `expire_in` setting does not affect:

- Artifacts from the latest job, unless keeping the latest job artifacts is disabled [at the project level](#) or [instance-wide](#).

After their expiry, artifacts are deleted hourly by default (using a cron job), and are not accessible anymore.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values: The expiry time. If no unit is provided, the time is in seconds. Valid values include:

- `'42'`

- `2 hrs 20 min`
- `2h20min`
- `6 mos 1 day`
- `47 yrs 6 mos and 4d`
- `3 weeks and 2 days`
- `never`

Example of `artifacts:expire_in:`

yaml

```

job:
  artifacts:
    expire_in: 1 week

```

Additional details:

- The expiration time period begins when the artifact is uploaded and stored on GitLab. If the expiry time is not defined, it defaults to the [instance wide setting](#).
- To override the expiration date and protect artifacts from being automatically deleted:
 - Select **Keep** on the job page.
 - Set the value of `expire_in` to `never`.
- If the expiry time is too short, jobs in later stages of a long pipeline might try to fetch expired artifacts from earlier jobs. If the artifacts are expired, jobs that try to fetch them fail with a [could not retrieve the needed artifacts error](#). Set the expiry time to be longer, or use [dependencies](#) in later jobs to ensure they don't try to fetch expired artifacts.
- `artifacts:expire_in` doesn't affect GitLab Pages deployments. To configure Pages deployments' expiry, use [pages.expire_in](#).

`artifacts:expose_as`

Use the `artifacts:expose_as` keyword to [expose artifacts in the merge request UI](#).

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- The name to display in the merge request UI for the artifacts download link. Must be combined with [artifacts:paths](#).

Example of `artifacts:expose_as:`

yaml

```

test:
  script: ["echo 'test' > file.txt"]
  artifacts:
    expose_as: 'artifact 1'
    paths: ['file.txt']

```

Additional details:

- You can use `expose_as` only once per job, with a maximum of 10 jobs per merge request.
- Glob patterns are not supported.
- Artifacts are always sent to GitLab. They are displayed in the UI unless `artifacts:paths` values:
 - Use [CI/CD variables](#).

- If `artifacts:paths` only includes a single file, the link opens the file directly. In all other cases, the link opens the [artifacts browser](#).
- Linked files are downloaded by default. If [GitLab Pages](#) is enabled, you can preview some artifacts file extensions directly in your browser. See [Browse the contents of the artifacts archive](#) for details.

Related topics:

- [Expose job artifacts in the merge request UI](#).

`artifacts:name`

Use the `artifacts:name` keyword to define the name of the created artifacts archive. You can specify a unique name for every archive.

If not defined, the default name is `artifacts`, which becomes `artifacts.zip` when downloaded.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- The name of the artifacts archive. CI/CD variables [are supported](#). Must be combined with `artifacts:paths`.

Example of `artifacts:name`:

To create an archive with a name of the current job:

yaml

```
job:
  artifacts:
    name: "job1-artifacts-file"
    paths:
      - binaries/
```

Related topics:

- [Use CI/CD variables to define the artifacts configuration](#)

`artifacts:public`

History >

 `artifacts:public` is now superseded by [artifacts:access](#) which has more options.

Use `artifacts:public` to control whether job artifacts in public pipelines are available for download with the GitLab UI and API by anonymous users, or Guest and Reporter roles.

 This option only affects GitLab UI and API access. CI/CD jobs using job tokens could still access artifacts with the runner API, regardless of this setting. To restrict job token access, configure your project's [CI/CD visibility settings](#) to **Only project members**.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- `true` (default): Artifacts in a job in public pipelines are available for download by anyone, including anonymous users, or Guest and Reporter roles.

yaml



```
job:
  artifacts:
    public: false
```

artifacts:access

History >

Use `artifacts:access` to determine who can access the job artifacts from the GitLab UI or API. This option does not prevent you from forwarding artifacts to downstream pipelines.

You cannot use `artifacts:public` and `artifacts:access` in the same job.

 This option only affects GitLab UI and API access. CI/CD jobs using job tokens could still access artifacts with the runner API, regardless of this setting. To restrict job token access, configure your project's [CI/CD visibility settings](#) to **Only project members**.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `all` (default): Artifacts in a job in public pipelines are available for download by anyone, including anonymous, guest, and reporter users.
- `developer`: Artifacts in the job are only available for download by users with the Developer, Maintainer, or Owner role.
- `maintainer`: Artifacts in the job are only available for download by users with the Maintainer or Owner role.
- `none`: Artifacts in the job are not available for download by anyone.

Example of `artifacts:access`:

yaml



```
job:
  artifacts:
    access: 'developer'
```

Additional details:

- `artifacts:access` affects all `artifacts:reports` too, so you can also restrict access to [artifacts for reports](#).

artifacts:reports

Use `artifacts:reports` to collect artifacts generated by included templates in jobs.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` [section](#).

Supported values:

- See list of available [artifacts reports types](#).

Example of `artifacts:reports`:

yaml



```
- bundle install
- rspec --format RspecJunitFormatter --out rspec.xml
artifacts:
  reports:
    junit: rspec.xml
```

Additional details:

- Combining reports in parent pipelines using [artifacts from child pipelines](#) is not supported. For more information, see [epic 8205](#).
- To be able to browse and download the report output files, include the `artifacts:paths` keyword. This uploads and stores the artifact twice.
- Artifacts created for `artifacts: reports` are always uploaded, regardless of the job results (success or failure). You can use `artifacts:expire_in` to set an expiration date for the artifacts.

artifacts:untracked

Use `artifacts:untracked` to add all Git untracked files as artifacts (along with the paths defined in `artifacts:paths`). `artifacts:untracked` ignores configuration in the repository's `.gitignore`, so matching artifacts in `.gitignore` are included.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- `true` or `false` (default if not defined).

Example of artifacts:untracked:

Save all Git untracked files:

yaml

```
job:
  artifacts:
    untracked: true
```

Related topics:

- [Add untracked files to artifacts](#).

artifacts:when

Use `artifacts:when` to upload artifacts on job failure or despite the failure.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- `on_success` (default): Upload artifacts only when the job succeeds.
- `on_failure`: Upload artifacts only when the job fails.
- `always`: Always upload artifacts (except when jobs time out). For example, when [uploading artifacts](#) required to troubleshoot failing tests.

Example of artifacts:when:

yaml

Additional details:

- The artifacts created for `artifacts:reports` are always uploaded, regardless of the job results (success or failure). `artifacts:when` does not change this behavior.

before_script

Use `before_script` to define an array of commands that should run before each job's `script` commands, but after [artifacts](#) are restored.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` [section](#).

Supported values: An array including:

- Single line commands.
- Long commands [split over multiple lines](#).
- [YAML anchors](#).

CI/CD variables [are supported](#).

Example of `before_script`:

yaml

```
job:
  before_script:
    - echo "Execute this command before any 'script:' commands."
  script:
    - echo "This command executes after the job's 'before_script' commands."
```

Additional details:

- Scripts you specify in `before_script` are concatenated with any scripts you specify in the main `script`. The combined scripts execute together in a single shell.
- Using `before_script` at the top level, but not in the `default` section, [is deprecated](#).

Related topics:

- [Use `before\_script` with `default`](#) to define a default array of commands that should run before the `script` commands in all jobs.
 - Job configuration and default configuration does not merge together. If the pipeline has `default:before_script` defined, and the job also has `before_script`, the job configuration takes precedence and the default configuration is not used.
- You can [ignore non-zero exit codes](#).
- [Use color codes with `before\_script`](#) to make job logs easier to review.
- [Create custom collapsible sections](#) to simplify job log output.

cache

History >

Use `cache` to specify a list of files and directories to cache between jobs. You can only use paths that are in the local working copy.

Caches are:

- Restored before [artifacts](#).
- Limited to a maximum of four [different caches](#).

You can [disable caching for specific jobs](#), for example to override:

- A default cache defined with `default`.
- The configuration for a job added with `include`.

Job configuration and default configuration does not merge together. If the pipeline has `default:cache` defined, and the job also has `cache`, the job configuration takes precedence and the default configuration is not used.

For more information about caches, see [Caching in GitLab CI/CD](#).

Using `cache` at the top level, but not in the `default` section, is [deprecated](#).

cache:paths

Use the `cache:paths` keyword to choose which files or directories to cache.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

- An array of paths relative to the project directory (`$CI_PROJECT_DIR`). You can use wildcards that use [glob](#) and [doublestar.glob](#) patterns.

[CI/CD variables](#) are supported.

Example of `cache:paths`:

Cache all files in `binaries` that end in `.apk` and the `.config` file:

yaml

```

rspec:
  script:
    - echo "This job uses a cache."
  cache:
    key: binaries-cache
    paths:
      - binaries/*.apk
      - .config

```

Additional details:

- The `cache:paths` keyword includes files even if they are untracked or in your `.gitignore` file.

Related topics:

- See the [CI/CD caching examples](#) for more `cache:paths` examples.

cache:key

Use the `cache:key` keyword to give each cache a unique identifying key. All jobs that use the same cache key use the same cache, including in different pipelines.

If not set, the default key is `default`. All jobs with the `cache` keyword but no `cache:key` share the `default` cache.

Must be used with `cache: paths`, or nothing is cached.

- A string.
- A predefined [CI/CD variable](#).
- A combination of both.

Example of `cache:key`:

yaml

```
cache-job:
  script:
    - echo "This job uses a cache."
  cache:
    key: binaries-cache-$CI_COMMIT_REF_SLUG
    paths:
      - binaries/
```

Additional details:

- If you use **Windows Batch** to run your shell scripts you must replace `$` with `%`. For example: `key: %CI_COMMIT_REF_SLUG%`
- The `cache:key` value can't contain:
 - The `/` character, or the equivalent URI-encoded `%2F`.
 - Only the `.` character (any number), or the equivalent URI-encoded `%2E`.
- The cache is shared between jobs, so if you're using different paths for different jobs, you should also set a different `cache:key`. Otherwise cache content can be overwritten.

Related topics:

- You can specify a [fallback cache key](#) to use if the specified `cache:key` is not found.
- You can [use multiple cache keys](#) in a single job.
- See the [CI/CD caching examples](#) for more `cache:key` examples.

`cache:key:files`

Use `cache:key:files` to generate a new cache key when the content of the specified files change. If the content remains unchanged, the cache key remains consistent across branches and pipelines. You can reuse caches and rebuild them less often, which speeds up subsequent pipeline runs.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- An array of up to two file paths or patterns.

CI/CD variables are not supported.

Example of `cache:key:files`:

yaml

```
cache:
  key:
    files:
      - Gemfile.lock
      - package.json
  paths:
    - vendor/ruby
    - node_modules
```

This example creates a cache for Ruby and Node.js dependencies. The cache is tied to the current versions of the `Gemfile.lock` and `package.json` files. When one of these files changes, a new cache key is computed and a new cache is created. Any future job runs that use the same `Gemfile.lock` and `package.json` with `cache:key:files` use the new cache, instead of rebuilding the dependencies.

Additional details:

- The cache `key` is a SHA computed from the content of the listed files. If a file doesn't exist, it's ignored in the key calculation. If none of the specified files exist, the fallback key is `default`.
- Wildcard patterns like `**/package.json` can be used.
- A maximum of two files can be specified. For updates on increasing the number of allowed paths or patterns, see [issue 301161](#).

`cache:key:files_commits`

Use `cache:key:files_commits` to generate a new cache key when the latest commit changes for the specified files. `cache:key:files_commits` cache keys change whenever the specified files have a new commit, even if the file content remains identical.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- An array of up to two file paths or patterns.

Example of `cache:key:files_commits`:

yaml

```
cache-job:
  script:
    - echo "This job uses a commit-based cache."
  cache:
    key:
      files_commits:
        - package.json
        - yarn.lock
  paths:
    - node_modules
```

This example creates a cache based on the commit history of `package.json` and `yarn.lock`. If the commit history changes for these files, a new cache key is computed and a new cache is created.

Additional details:

- The cache `key` is a SHA computed from the most recent commit for each specified file.
- If a file doesn't exist, it's ignored in the key calculation.
- If none of the specified files exist, the fallback key is `default`.
- Cannot be used together with [cache:key:files](#) in the same cache configuration.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

- A string.
- A predefined [CI/CD variable](#).
- A combination of both.

Example of `cache:key:prefix:`

yaml

```
rspec:
  script:
    - echo "This rspec job uses a cache."
  cache:
    key:
      files:
        - Gemfile.lock
      prefix: $CI_JOB_NAME
    paths:
      - vendor/ruby
```

For example, adding a `prefix` of `$CI_JOB_NAME` causes the key to look like `rspec-fee9576d21ee9b6a32e30c5c79d0a0ceb68d1e5`. If a branch changes `Gemfile.lock`, that branch has a new SHA checksum for `cache:key:files`. A new cache key is generated, and a new cache is created for that key. If `Gemfile.lock` is not found, the prefix is added to `default`, so the key in the example would be `rspec-default`.

Additional details:

- If no file in `cache:key:files` is changed in any commits, the prefix is added to the `default` key.

`cache:untracked`

Use `untracked: true` to cache all files that are untracked in your Git repository. Untracked files include files that are:

- Ignored due to [.gitignore configuration](#).
- Created, but not added to the checkout with `git add`.

Caching untracked files can create unexpectedly large caches if the job downloads:

- Dependencies, like gems or node modules, which are usually untracked.
- [Artifacts](#) from a different job. Files extracted from the artifacts are untracked by default.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

- `true` or `false` (default).

Example of `cache:untracked:`

yaml

```
rspec:
  script: test
  cache:
    untracked: true
```

Additional details:

`cache:` is intended to also cache all untracked files. For example:

yaml

```
rspec:
  script: test
  cache:
    untracked: true
    paths:
      - binaries/
```

In this example, the job caches all untracked files in the repository, as well as all the files in `binaries/`. If there are untracked files in `binaries/`, they are covered by both keywords.

cache:unprotect

History >

Use `cache:unprotect` to set a cache to be shared between [protected](#) and unprotected branches.

⚠ When set to `true`, users without access to protected branches can read and write to cache keys used by protected branches.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- `true` or `false` (default).

Example of `cache:unprotect`:

yaml

```
rspec:
  script: test
  cache:
    unprotect: true
```

cache:when

Use `cache:when` to define when to save the cache, based on the status of the job.

Must be used with `cache: paths`, or nothing is cached.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- `on_success` (default): Save the cache only when the job succeeds.
- `on_failure`: Save the cache only when the job fails.
- `always`: Always save the cache.

Example of `cache:when`:

yaml

```
paths:
  - rspec/
when: 'always'
```

This example stores the cache whether or not the job fails or succeeds.

cache:policy

To change the upload and download behavior of a cache, use the `cache:policy` keyword. By default, the job downloads the cache when the job starts, and uploads changes to the cache when the job ends. This caching style is the `pull-push` policy (default).

To set a job to only download the cache when the job starts, but never upload changes when the job finishes, use `cache:policy:pull`.

To set a job to only upload a cache when the job finishes, but never download the cache when the job starts, use `cache:policy:push`.

Use the `pull` policy when you have many jobs executing in parallel that use the same cache. This policy speeds up job execution and reduces load on the cache server. You can use a job with the `push` policy to build the cache.

Must be used with `cache: paths`, or nothing is cached.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- `pull`
- `push`
- `pull-push` (default)
- [CI/CD variables](#).

Example of cache:policy:

yaml



```
prepare-dependencies-job:
  stage: build
  cache:
    key: gems
    paths:
      - vendor/bundle
    policy: push
  script:
    - echo "This job only downloads dependencies and builds the cache."
    - echo "Downloading dependencies..."

faster-test-job:
  stage: test
  cache:
    key: gems
    paths:
      - vendor/bundle
    policy: pull
  script:
    - echo "This job script uses the cache, but does not update it."
    - echo "Running tests..."
```

Related topics:

cache: fallback_keys

Use `cache: fallback_keys` to specify a list of keys to try to restore cache from if there is no cache found for the `cache: key`. Caches are retrieved in the order specified in the `fallback_keys` section.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- An array of cache keys

Example of `cache: fallback_keys`:

yaml


```

rspec:
  script: rspec
  cache:
    key: gems-$CI_COMMIT_REF_SLUG
    paths:
      - rspec/
    fallback_keys:
      - gems
  when: 'always'

```

coverage

Use `coverage` with a custom regular expression to configure how code coverage is extracted from the job output. The coverage is shown in the UI if at least one line in the job output matches the regular expression.

To extract the code coverage value from the match, GitLab uses this smaller regular expression: `\d+(?:\.\d+)?`.

Supported values:

- An RE2 regular expression. Must start and end with `/`. Must match the coverage number. May match surrounding text as well, so you don't need to use a regular expression character group to capture the exact number. Because it uses RE2 syntax, all groups must be non-capturing.

Example of `coverage`:

yaml


```

job1:
  script: rspec
  coverage: '/Code coverage: \d+(?:\.\d+)?/'

```

In this example:

1. GitLab checks the job log for a match with the regular expression. A line like `Code coverage: 67.89% of lines covered` would match.
2. GitLab then checks the matched fragment to find a match to the regular expression: `\d+(?:\.\d+)?`. The sample regex can match a code coverage of `67.89`.

Additional details:

- You can find regex examples in [Code Coverage](#).
- If there is more than one matched line in the job output, the last line is used (the first result of reverse search).
- If there are multiple matches in a single line, the last match is searched for the coverage number.
- If there are multiple coverage numbers found in the matched fragment, the first number is used.

dast_configuration

Tier: Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

Use the `dast_configuration` keyword to specify a site profile and scanner profile to be used in a CI/CD configuration. Both profiles must first have been created in the project. The job's stage must be `dast`.

Keyword type: Job keyword. You can use only as part of a job.

Supported values: One each of `site_profile` and `scanner_profile`.

- Use `site_profile` to specify the site profile to be used in the job.
- Use `scanner_profile` to specify the scanner profile to be used in the job.

Example of `dast_configuration`:

yaml

```
stages:
  - build
  - dast

include:
  - template: DAST.gitlab-ci.yml

dast:
  dast_configuration:
    site_profile: "Example Co"
    scanner_profile: "Quick Passive Test"
```

In this example, the `dast` job extends the `dast` configuration added with the `include` keyword to select a specific site profile and scanner profile.

Additional details:

- Settings contained in either a site profile or scanner profile take precedence over those contained in the DAST template.

Related topics:

- [Site profile](#).
- [Scanner profile](#).

dependencies

Use the `dependencies` keyword to define a list of specific jobs to fetch [artifacts](#) from. The specified jobs must all be in earlier stages. You can also set a job to download no artifacts at all.

When `dependencies` is not defined in a job, all jobs in earlier stages are considered dependent and the job fetches all artifacts from those jobs.

To fetch artifacts from a job in the same stage, you must use `needs:artifacts`. You should not combine `dependencies` with `needs` in the same job.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

Example of dependencies:

yaml

```
build osx:
  stage: build
  script: make build:osx
  artifacts:
    paths:
      - binaries/

build linux:
  stage: build
  script: make build:linux
  artifacts:
    paths:
      - binaries/

test osx:
  stage: test
  script: make test:osx
  dependencies:
    - build osx

test linux:
  stage: test
  script: make test:linux
  dependencies:
    - build linux

deploy:
  stage: deploy
  script: make deploy
  environment: production
```

In this example, two jobs have artifacts: `build osx` and `build linux`. When `test osx` is executed, the artifacts from `build osx` are downloaded and extracted in the context of the build. The same thing happens for `test linux` and artifacts from `build linux`.

The `deploy` job downloads artifacts from all previous jobs because of the [stage](#) precedence.

Additional details:

- The job status does not matter. If a job fails or it's a manual job that isn't triggered, no error occurs.
- If the artifacts of a dependent job are [expired](#) or [deleted](#), then the job fails.

environment

Use `environment` to define the [environment](#) that a job deploys to.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: The name of the environment the job deploys to, in one of these formats:

- Plain text, including letters, digits, spaces, and these characters: `-`, `_`, `/`, `$`, `{`, `}`.
- CI/CD variables, including predefined, project, group, instance, or variables defined in the `.gitlab-ci.yml` file. You can't use variables defined in a `script` section.

Example of environment:

```
stage: deploy
script: git push production HEAD:main
environment: production
```

Additional details:

- If you specify an `environment` and no environment with that name exists, an environment is created.

`environment:name`

Set a name for an [environment](#).

Common environment names are `qa`, `staging`, and `production`, but you can use any name.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: The name of the environment the job deploys to, in one of these formats:

- Plain text, including letters, digits, spaces, and these characters: `-`, `_`, `/`, `$`, `{`, `}`.
- [CI/CD variables](#), including predefined, project, group, instance, or variables defined in the `.gitlab-ci.yml` file. You can't use variables defined in a `script` section.

Example of `environment:name`:

`yaml`

```
deploy to production:
  stage: deploy
  script: git push production HEAD:main
  environment:
    name: production
```

`environment:url`

Set a URL for an [environment](#).

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: A single URL, in one of these formats:

- Plain text, like `https://prod.example.com`.
- [CI/CD variables](#), including predefined, project, group, instance, or variables defined in the `.gitlab-ci.yml` file. You can't use variables defined in a `script` section.

Example of `environment:url`:

`yaml`

```
deploy to production:
  stage: deploy
  script: git push production HEAD:main
  environment:
    name: production
    url: https://prod.example.com
```

Additional details:

- After the job completes, you can access the URL by selecting a button in the merge request, environment, or deployment pages.

Closing (stopping) environments can be achieved with the `on_stop` keyword defined under `environment`. It declares a different job that runs to close the environment.

Keyword type: Job keyword. You can use it only as part of a job.

Additional details:

- See `environment:action` for more details and an example.

`environment:action`

Use the `action` keyword to specify how the job interacts with the environment.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: One of the following keywords:

Value	Description
<code>start</code>	Default value. Indicates that the job starts the environment. The deployment is created after the job starts.
<code>prepare</code>	Indicates that the job is only preparing the environment. It does not trigger deployments. Read more about preparing environments .
<code>stop</code>	Indicates that the job stops an environment. Read more about stopping an environment .
<code>verify</code>	Indicates that the job is only verifying the environment. It does not trigger deployments. Read more about verifying environments .
<code>access</code>	Indicates that the job is only accessing the environment. It does not trigger deployments. Read more about accessing environments .

Example of `environment:action`:

`yml`

```
stop_review_app:
  stage: deploy
  variables:
    GIT_STRATEGY: none
  script: make delete-app
  when: manual
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
```

`environment:auto_stop_in`

History >

The `auto_stop_in` keyword specifies the lifetime of the environment. When an environment expires, GitLab automatically stops it.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: A period of time written in natural language. For example, these are all equivalent:

- `one week`
- `never`

CI/CD variables [are supported](#).

Example of `environment:auto_stop_in`:

yaml

```
review_app:
  script: deploy-review-app
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    auto_stop_in: 1 day
```

When the environment for `review_app` is created, the environment’s lifetime is set to `1 day`. Every time the review app is deployed, that lifetime is also reset to `1 day`.

The `auto_stop_in` keyword can be used for all [environment actions](#) except `stop`. Some actions can be used to reset the scheduled stop time for the environment. For more information, see [Access an environment for preparation or verification purposes](#).

Related topics:

- [Environments auto-stop documentation](#).

environment:kubernetes

History >

Use the `kubernetes` keyword to configure the [dashboard for Kubernetes](#) and [GitLab-managed Kubernetes resources](#) for an environment.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `agent`: A string specifying the [GitLab agent for Kubernetes](#). The format is `path/to/agent/project:agent-name`. If the agent is connected to the project running the pipeline, use `$CI_PROJECT_PATH:agent-name`.
- `dashboard:namespace`: A string representing the Kubernetes namespace where the environment is deployed. The namespace must be set together with the `agent` keyword. `namespace` is [deprecated](#).
- `dashboard:flux_resource_path`: A string representing the full path to the Flux resource, such as a `HelmRelease`. The Flux resource must be set together with the `agent` and `dashboard:namespace` keywords. `flux_resource_path` is [deprecated](#).
- `managed_resources`: A hash with the `enabled` keyword to configure the [GitLab-managed Kubernetes resources](#) for the environment.
 - `managed_resources:enabled`: A boolean value indicating whether GitLab-managed Kubernetes resources are enabled for the environment.
- `dashboard`: A hash with the `dashboard:namespace` and `dashboard:flux_resource_path` keywords to configure the [dashboard for Kubernetes](#) for the environment.

Example of `environment:kubernetes`:

yaml

```
environment:
  name: production
  kubernetes:
    agent: path/to/agent/project:agent-name
    dashboard:
      namespace: my-namespace
      flux_resource_path: helm.toolkit.fluxcd.io/v2/namespaces/flux-system/helmreleases/helm-release-resource
```

Example of `environment:kubernetes` when disabling managed resources:

yaml

```
deploy:
  stage: deploy
  script: make deploy-app
  environment:
    name: production
    kubernetes:
      agent: path/to/agent/project:agent-name
      managed_resources:
        enabled: false
      dashboard:
        namespace: my-namespace
        flux_resource_path: helm.toolkit.fluxcd.io/v2/namespaces/flux-system/helmreleases/helm-release-resource
```

This configuration:

- Sets up the `deploy` job to deploy to the `production` environment.
- Associates the [agent](#) named `agent-name` with the environment.
- Configures the [dashboard for Kubernetes](#) for an environment with the namespace `my-namespace` and the `flux_resource_path` set to `helm.toolkit.fluxcd.io/v2/namespaces/flux-system/helmreleases/helm-release-resource`.

Additional details:

- To use the dashboard, you must [install the GitLab agent for Kubernetes](#) and [configure user_access](#) for the environment’s project or its parent group.
- The user running the job must be authorized to access the cluster agent. Otherwise, the dashboard ignores the `agent`, `namespace`, and `flux_resource_path` attributes.
- If you only want to set the `agent`, you do not have to set the `namespace`, and cannot set `flux_resource_path`. However, this configuration lists all namespaces in a cluster in the dashboard for Kubernetes.

`environment:deployment_tier`

History >

Use the `deployment_tier` keyword to specify the tier of the deployment environment.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: One of the following:

- `production`
- `staging`
- `testing`
- `development`

variables defined in a `script` section.

Example of `environment:deployment_tier:`

yaml

```
deploy:
  script: echo
  environment:
    name: customer-portal
    deployment_tier: production
```

Additional details:

- Environments created from this job definition are assigned a [tier](#) based on this value.
- Existing environments don't have their tier updated if this value is added later. Existing environments must have their tier updated via the [Environments API](#).

Related topics:

- [Deployment tier of environments](#).

Dynamic environments

Use CI/CD [variables](#) to dynamically name environments.

For example:

yaml

```
deploy as review app:
  stage: deploy
  script: make deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_ENVIRONMENT_SLUG.example.com/
```

The `deploy as review app` job is marked as a deployment to dynamically create the `review/$CI_COMMIT_REF_SLUG` environment. `$CI_COMMIT_REF_SLUG` is a [CI/CD variable](#) set by the runner. The `$CI_ENVIRONMENT_SLUG` variable is based on the environment name, but suitable for inclusion in URLs. If the `deploy as review app` job runs in a branch named `pow`, this environment would be accessible with a URL like `https://review-pow.example.com/`.

The common use case is to create dynamic environments for branches and use them as review apps. You can see an example that uses review apps at <https://gitlab.com/gitlab-examples/review-apps-nginx/>.

extends

Use `extends` to reuse configuration sections. It's an alternative to [YAML anchors](#) and is a little more flexible and readable.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- The name of another job in the pipeline.
- A list (array) of names of other jobs in the pipeline.

Example of `extends:`

yaml

```
rspec:
  extends: .tests
  script: rake rspec

rubocop:
  extends: .tests
  script: bundle exec rubocop
```

In this example, the `rspec` job uses the configuration from the `.tests` template job. When creating the pipeline, GitLab:

- Performs a reverse deep merge based on the keys.
- Merges the `.tests` content with the `rspec` job.
- Doesn't merge the values of the keys.

The combined configuration is equivalent to these jobs:

yaml


```
rspec:
  stage: test
  image: ruby:3.0
  script: rake rspec

rubocop:
  stage: test
  image: ruby:3.0
  script: bundle exec rubocop
```

Additional details:

- You can use multiple parents for `extends`.
- The `extends` keyword supports up to eleven levels of inheritance, but you should avoid using more than three levels.
- In the previous example, `.tests` is a [hidden job](#), but you can extend configuration from regular jobs as well.

Related topics:

- [Reuse configuration sections by using `extends`](#).
- Use `extends` to reuse configuration from [included configuration files](#).

hooks

History >

Use `hooks` to specify lists of commands to execute on the runner at certain stages of job execution, like before retrieving the Git repository.

Job configuration and default configuration does not merge together. If the pipeline has `default:hooks` defined, and the job also has `hooks`, the job configuration takes precedence and the default configuration is not used.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- A hash of hooks and their commands. Available hooks: `pre_get_sources_script`.

```
hooks:pre_get_sources_script
```

Use `hooks:pre_get_sources_script` to specify a list of commands to execute on the runner before cloning the Git repository and any submodules. You can use it for example to:

- Adjust the [Git configuration](#).
- Export [tracing variables](#).

Supported values: An array including:

- Single line commands.
- Long commands [split over multiple lines](#).
- [YAML anchors](#).

CI/CD variables [are supported](#).

Example of `hooks:pre_get_sources_script`:

`yaml`

```
job1:
  hooks:
    pre_get_sources_script:
      - echo 'hello job1 pre_get_sources_script'
  script: echo 'hello job1 script'
```



Related topics:

- [GitLab Runner configuration](#)

identity

Tier: Free, Premium, Ultimate

Offering: GitLab.com

Status: Beta

History >

This feature is in [beta](#).

Use `identity` to authenticate with third party services using identity federation.

Keyword type: Job keyword. You can use it only as part of a job or in the `default:` [section](#).

Supported values: An identifier. Supported providers:

- `google_cloud`: Google Cloud. Must be configured with the [Google Cloud IAM integration](#).

Example of `identity`:

`yaml`

```
job_with_workload_identity:
  identity: google_cloud
  script:
    - gcloud compute instances list
```



Related topics:

- [Workload Identity Federation](#) 
- [Google Cloud IAM integration](#).

Use `id_tokens` to create [ID tokens](#) to authenticate with third party services. All JWTs created this way support OIDC authentication. The required `aud` sub-keyword is used to configure the `aud` claim for the JWT.

Job configuration and default configuration does not merge together. If the pipeline has `default:id_tokens` defined, and the job also has `id_tokens`, the job configuration takes precedence and the default configuration is not used.

Supported values:

- Token names with their `aud` claims. `aud` supports:
 - A single string.
 - An array of strings.
 - [CI/CD variables](#).

Example of `id_tokens`:

yaml

```
job_with_id_tokens:
  id_tokens:
    ID_TOKEN_1:
      aud: https://vault.example.com
    ID_TOKEN_2:
      aud:
        - https://gcp.com
        - https://aws.com
    SIGSTORE_ID_TOKEN:
      aud: sigstore
  script:
    - command_to_authenticate_with_vault $ID_TOKEN_1
    - command_to_authenticate_with_aws $ID_TOKEN_2
    - command_to_authenticate_with_gcp $ID_TOKEN_2
```

Related topics:

- [ID token authentication](#).
- [Connect to cloud services](#).
- [Keyless signing with Sigstore](#).

image

Use `image` to specify a Docker image that the job runs in.

Job configuration and default configuration does not merge together. If the pipeline has `default:image` defined, and the job also has `image`, the job configuration takes precedence and the default configuration is not used.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values: The name of the image, including the registry path if needed, in one of these formats:

- `<image-name>` (Same as using `<image-name>` with the `latest` tag)
- `<image-name>:<tag>`
- `<image-name>@<digest>`

CI/CD variables [are supported](#).

Example of `image`:


```
image: ruby:3.0

rspec:
  script: bundle exec rspec

rspec 2.7:
  image: registry.example.com/my-group/my-project/ruby:2.7
  script: bundle exec rspec
```

In this example, the `ruby:3.0` image is the default for all jobs in the pipeline. The `rspec 2.7` job does not use the default, because it overrides the default with a job-specific `image` section.

Additional details:

- Using `image` at the top level, but not in the `default` section, is [deprecated](#).

Related topics:

- [Run your CI/CD jobs in Docker containers.](#)

image:name

The name of the Docker image that the job runs in. Similar to [image](#) used by itself.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values: The name of the image, including the registry path if needed, in one of these formats:

- `<image-name>` (Same as using `<image-name>` with the `latest` tag)
- `<image-name>:<tag>`
- `<image-name>@<digest>`

CI/CD variables [are supported](#).

Example of `image:name`:

yaml

```
test-job:
  image:
    name: "registry.example.com/my/image:latest"
  script: echo "Hello world"
```

Related topics:

- [Run your CI/CD jobs in Docker containers.](#)

image:entrypoint

Command or script to execute as the container’s entry point.

When the Docker container is created, the `entrypoint` is translated to the Docker `--entrypoint` option. The syntax is similar to the [Dockerfile ENTRYPOINT directive](#), where each shell token is a separate string in the array.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- A string.

yaml



```
test-job:
  image:
    name: super/sql:experimental
    entrypoint: [""]
  script: echo "Hello world"
```

Related topics:

- [Override the entrypoint of an image.](#)

image:docker

History >

Use `image:docker` to pass options to runners using the [Docker executor](#) or the [Kubernetes executor](#). This keyword does not work with other executor types.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` [section](#).

Supported values:

A hash of options for the Docker executor, which can include:

- `platform`: Selects the architecture of the image to pull. When not specified, the default is the same platform as the host runner.
- `user`: Specify the username or UID to use when running the container.

Example of `image:docker`:

yaml



```
arm-sql-job:
  script: echo "Run sql tests"
  image:
    name: super/sql:experimental
    docker:
      platform: arm64/v8
      user: dave
```

Additional details:

- `image:docker:platform` maps to the `docker pull --platform` [option](#)
- `image:docker:user` maps to the `docker run --user` [option](#)

image:kubernetes

History >

Use `image:kubernetes` to pass options to the GitLab Runner [Kubernetes executor](#). This keyword does not work with other executor types.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` [section](#).

Supported values:

A hash of options for the Kubernetes executor, which can include:

Example of `image:kubernetes` with only UID:

yaml

```
arm-sql-job:
  script: echo "Run sql tests"
  image:
    name: super/sql:experimental
    kubernetes:
      user: "1001"
```

Example of `image:kubernetes` with both UID and GID:

yaml

```
arm-sql-job:
  script: echo "Run sql tests"
  image:
    name: super/sql:experimental
    kubernetes:
      user: "1001:1001"
```

`image:pull_policy`

History >

The pull policy that the runner uses to fetch the Docker image.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

- A single pull policy, or multiple pull policies in an array. Can be `always`, `if-not-present`, or `never`.

Examples of `image:pull_policy`:

yaml

```
job1:
  script: echo "A single pull policy."
  image:
    name: ruby:3.0
    pull_policy: if-not-present

job2:
  script: echo "Multiple pull policies."
  image:
    name: ruby:3.0
    pull_policy: [always, if-not-present]
```

Additional details:

- If the runner does not support the defined pull policy, the job fails with an error similar to: `ERROR: Job failed (system failure): the configured PullPolicies ([always]) are not allowed by AllowedPullPolicies ([never])`.

Related topics:

- [Run your CI/CD jobs in Docker containers.](#)
- [Configure how runners pull images.](#)
- [Set multiple pull policies.](#)

History >

Use `inputs` to define typed and validated inputs for a job. [Job inputs](#) can be overridden when manually running or retrying a job.

Job inputs are parameters that provide type safety and validation. Unlike [CI/CD variables](#), only inputs explicitly defined in the job can be specified when running or retrying the job. All job input names must be predefined.

Reference job input values with the `${{ job.inputs.INPUT_NAME }}` [Moa expression](#) syntax.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

A hash of input names, where each input is configured with one or more subkeys:

- `default` (required)
- `type`
- `options`
- `description`
- `regex`

Example of `inputs`:

yaml

```
test_job:
  inputs:
    test_suite:
      default: unit
      description: Which test suite to run
      options: [unit, integration, e2e]
    parallel_count:
      type: number
      default: 5
      description: Number of parallel test runners
    verbose:
      type: boolean
      default: false
      description: Enable verbose test output
    script:
      - 'echo "Running ${{ job.inputs.test_suite }} tests"'
      - 'if [ "${{ job.inputs.verbose }}" == "true" ]; then export TEST_VERBOSE=1; fi'
      - './run_tests.sh --suite ${{ job.inputs.test_suite }} --parallel ${{ job.inputs.parallel_count }}
```

Additional details:

- Job inputs are validated when the job is created and when you try to retry a job with new input values. If validation fails, the job does not start.
- Job inputs are scoped to the job where they are defined and cannot be accessed by other jobs.
- For a complete list of keywords that support job inputs, see [where you can use job inputs](#).

`inputs:default`

All job inputs must have a default value defined with `default`.

Keyword type: Job keyword. You can use it only as part of a job.

yaml



```
test_job:
  inputs:
    environment:
      default: staging
    timeout:
      type: number
      default: 30
```

inputs:type

Use `type` to define the data type of the input value.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `string` (default)
- `number`
- `boolean`
- `array`.

Example of `inputs:type`:

yaml



```
test_job:
  inputs:
    count:
      type: number
      default: 5
    enabled:
      type: boolean
      default: true
```

inputs:description

Use `description` to provide information about the input's purpose. The description does not affect the input's behavior.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: A string.

Example of `inputs:description`:

yaml



```
deploy_job:
  inputs:
    environment:
      default: staging
    description: Target deployment environment
```

inputs:options

option.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: An array of allowed values.

Example of `inputs:options:`

yaml

```
deploy_job:
  inputs:
    environment:
      default: staging
      options: [development, staging, production]
```

`inputs:regex`

Use `regex` to specify a regular expression pattern that the input value must match.

Validation fails if the value does not match the regular expression.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: A regular expression string.

Example of `inputs:regex:`

yaml

```
deploy_job:
  inputs:
    version:
      default: v1.0.0
      regex: ^v\d+\.\d+\.\d+$
```

In this example, an input value of `v1.1.1` passes the regex validation, but an input of `v1.1.1-beta` does not.

`inherit`

Use `inherit` to [control inheritance of default keywords and variables](#).

`inherit:default`

Use `inherit:default` to control the inheritance of [default keywords](#).

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `true` (default) or `false` to enable or disable the inheritance of all default keywords.
- A list of specific default keywords to inherit.

Example of `inherit:default:`

yaml

```
interruptible: true

job1:
  script: echo "This job does not inherit any default keywords."
  inherit:
    default: false

job2:
  script: echo "This job inherits only the two listed default keywords. It does not inherit
'interruptible'."
  inherit:
    default:
      - retry
      - image
```

Additional details:

- You can also list default keywords to inherit on one line: `default: [keyword1, keyword2]`

inherit:variables

Use `inherit:variables` to control the inheritance of [default variables](#) keywords.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `true` (default) or `false` to enable or disable the inheritance of all default variables.
- A list of specific variables to inherit.

Example of `inherit:variables`:

yaml

```
variables:
  VARIABLE1: "This is default variable 1"
  VARIABLE2: "This is default variable 2"
  VARIABLE3: "This is default variable 3"

job1:
  script: echo "This job does not inherit any default variables."
  inherit:
    variables: false

job2:
  script: echo "This job inherits only the two listed default variables. It does not inherit 'VARIABLE3'."
  inherit:
    variables:
      - VARIABLE1
      - VARIABLE2
```

Additional details:

- You can also list default variables to inherit on one line: `variables: [VARIABLE1, VARIABLE2]`

interruptible

be for a commit with new changes. For example, the **Auto-cancel redundant pipelines** feature has no effect if you select **New pipeline** in the UI to run a pipeline for the same commit.

The behavior of the **Auto-cancel redundant pipelines** feature can be controlled by the `workflow:auto_cancel:on_new_commit` setting.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

- `true` or `false` (default).

Example of `interruptible` with the default behavior:

yaml

```
workflow:
  auto_cancel:
    on_new_commit: conservative # the default behavior

stages:
  - stage1
  - stage2
  - stage3

step-1:
  stage: stage1
  script:
    - echo "Can be canceled."
  interruptible: true

step-2:
  stage: stage2
  script:
    - echo "Can not be canceled."

step-3:
  stage: stage3
  script:
    - echo "Because step-2 can not be canceled, this step can never be canceled, even though it's set as interruptible."
  interruptible: true
```

In this example, a new pipeline causes a running pipeline to be:

- Canceled, if only `step-1` is running or pending.
- Not canceled, after `step-2` starts.

Example of `interruptible` with the `auto_cancel:on_new_commit:interruptible` setting:

yaml


```

stages:
  - stage1
  - stage2
  - stage3

step-1:
  stage: stage1
  script:
    - echo "Can be canceled."
  interruptible: true

step-2:
  stage: stage2
  script:
    - echo "Can not be canceled."

step-3:
  stage: stage3
  script:
    - echo "Can be canceled."
  interruptible: true

```

In this example, a new pipeline causes a running pipeline to cancel `step-1` and `step-3` if they are running or pending.

Additional details:

- Only set `interruptible: true` if the job can be safely canceled after it has started, like a build job. Deployment jobs usually shouldn't be canceled, to prevent partial deployments.
- When using the default behavior or `workflow:auto_cancel:on_new_commit: conservative`:
 - A job that has not started yet is always considered `interruptible: true`, regardless of the job's configuration. The `interruptible` configuration is only considered after the job starts.
 - Running** pipelines are only canceled if all running jobs are configured with `interruptible: true` or no jobs configured with `interruptible: false` have started at any time. After a job with `interruptible: false` starts, the entire pipeline is no longer considered interruptible.
 - If the pipeline triggered a downstream pipeline, but no job with `interruptible: false` in the downstream pipeline has started yet, the downstream pipeline is also canceled.
- You can add an optional manual job with `interruptible: false` in the first stage of a pipeline to allow users to manually prevent a pipeline from being automatically canceled. After a user starts the job, the pipeline cannot be canceled by the **Auto-cancel redundant pipelines** feature.
- When using `interruptible` with a [trigger job](#):
 - The triggered downstream pipeline is never affected by the trigger job's `interruptible` configuration.
 - If `workflow:auto_cancel` is set to `conservative`, the trigger job's `interruptible` configuration has no effect.
 - If `workflow:auto_cancel` is set to `interruptible`, a trigger job with `interruptible: true` can be automatically canceled.

needs

Use `needs` to execute jobs out-of-order. Relationships between jobs that use `needs` can be visualized as a [directed acyclic graph](#).

You can ignore stage ordering and run some jobs without waiting for others to complete. Jobs in multiple stages can run concurrently.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

Example of `needs`:

yaml


```

linux:build:
  stage: build
  script: echo "Building linux..."

mac:build:
  stage: build
  script: echo "Building mac..."

lint:
  stage: test
  needs: []
  script: echo "Linting..."

linux:rspec:
  stage: test
  needs: ["linux:build"]
  script: echo "Running rspec on linux..."

mac:rspec:
  stage: test
  needs: ["mac:build"]
  script: echo "Running rspec on mac..."

production:
  stage: deploy
  script: echo "Running production..."
  environment: production
  
```

This example creates four paths of execution:

- Linter: The `lint` job runs immediately without waiting for the `build` stage to complete because it has no needs (`needs: []`).
- Linux path: The `linux:rspec` job runs as soon as the `linux:build` job finishes, without waiting for `mac:build` to finish.
- macOS path: The `mac:rspec` jobs runs as soon as the `mac:build` job finishes, without waiting for `linux:build` to finish.
- The `production` job runs as soon as all previous jobs finish: `lint`, `linux:build`, `linux:rspec`, `mac:build`, `mac:rspec`.

Additional details:

- The maximum number of jobs that a single job can have in the `needs` array is limited:
 - For GitLab.com, the limit is 50. For more information, see [issue 350398](#).
 - For GitLab Self-Managed and GitLab Dedicated, the default limit is 50. This limit can be changed by [updating the CI/CD limits in the Admin area](#).
- If `needs` refers to a job that uses the `parallel` keyword, it depends on all jobs created in parallel, not just one job. It also downloads artifacts from all the parallel jobs by default. If the artifacts have the same name, they overwrite each other and only the last one downloaded is saved.
 - To have `needs` refer to a subset of parallelized jobs (and not all of the parallelized jobs), use the `needs:parallel:matrix` keyword.
- You can refer to jobs in the same stage as the job you are configuring.
- If `needs` refers to a job that might not be added to a pipeline because of `only`, `except`, or `rules`, the pipeline might fail to create. Use the `needs:optional` keyword to resolve a failed pipeline creation.
- If a pipeline has jobs with `needs: []` and jobs in the `.pre` stage, they will all start as soon as the pipeline is created. Jobs with `needs: []` start immediately, and jobs in the `.pre` stage also start immediately.

before earlier stages complete. With `needs` you can only download artifacts from the jobs listed in the `needs` configuration.

Use `artifacts: true` (default) or `artifacts: false` to control when artifacts are downloaded in jobs that use `needs`.

Keyword type: Job keyword. You can use it only as part of a job. Must be used with `needs: job`.

Supported values:

- `true` (default) or `false`.

Example of `needs: artifacts`:

yaml

```
test-job1:
  stage: test
  needs:
    - job: build_job1
      artifacts: true

test-job2:
  stage: test
  needs:
    - job: build_job2
      artifacts: false

test-job3:
  needs:
    - job: build_job1
      artifacts: true
    - job: build_job2
    - build_job3
```

In this example:

- The `test-job1` job downloads the `build_job1` artifacts
- The `test-job2` job does not download the `build_job2` artifacts.
- The `test-job3` job downloads the artifacts from all three `build_jobs`, because `artifacts` is `true`, or defaults to `true`, for all three needed jobs.

Additional details:

- You should not combine `needs` with `dependencies` in the same job.

`needs: project`

Tier: Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

Use `needs: project` to download artifacts from up to five jobs in other pipelines. The artifacts are downloaded from the latest successful specified job for the specified ref. To specify multiple jobs, add each as separate array items under the `needs` keyword.

If there is a pipeline running for the ref, a job with `needs: project` does not wait for the pipeline to complete. Instead, the artifacts are downloaded from the latest successful run of the specified job.

`needs: project` must be used with `job`, `ref`, and `artifacts`.

Keyword type: Job keyword. You can use it only as part of a job.

- `job`: The job to download artifacts from.
- `ref`: The ref to download artifacts from.
- `artifacts`: Must be `true` to download artifacts.

Examples of `needs:project`:

yaml


```

build_job:
  stage: build
  script:
    - ls -lhR
  needs:
    - project: namespace/group/project-name
      job: build-1
      ref: main
      artifacts: true
    - project: namespace/group/project-name-2
      job: build-2
      ref: main
      artifacts: true

```

In this example, `build_job` downloads the artifacts from the latest successful `build-1` and `build-2` jobs on the `main` branches in the `group/project-name` and `group/project-name-2` projects.

You can use [CI/CD variables](#) in `needs:project`, for example:

yaml


```

build_job:
  stage: build
  script:
    - ls -lhR
  needs:
    - project: $CI_PROJECT_PATH
      job: $DEPENDENCY_JOB_NAME
      ref: $ARTIFACTS_DOWNLOAD_REF
      artifacts: true

```

Additional details:

- To download artifacts from a different pipeline in the current project, set `project` to be the same as the current project, but use a different ref than the current pipeline. Concurrent pipelines running on the same ref could override the artifacts.
- The user running the pipeline must have the Reporter, Developer, Maintainer, or Owner role for the group or project, or the group/project must have public visibility.
- You can't use `needs:project` in the same job as [trigger](#).
- When using `needs:project` to download artifacts from another pipeline, the job does not wait for the needed job to complete. [Using needs to wait for jobs to complete](#) is limited to jobs in the same pipeline. Make sure that the needed job in the other pipeline completes before the job that needs it tries to download the artifacts.
- You can't download artifacts from jobs that run in [parallel](#).
- Support [CI/CD variables](#) in `project`, `job`, and `ref`.

Related topics:

- To download artifacts between [parent-child pipelines](#), use [needs:pipeline:job](#).

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `needs: pipeline`: A pipeline ID. Must be a pipeline present in the same parent-child pipeline hierarchy.
- `job`: The job to download artifacts from.

Example of `needs: pipeline: job`:

- Parent pipeline (`.gitlab-ci.yml`):

yaml


```

stages:
  - build
  - test

create-artifact:
  stage: build
  script: echo "sample artifact" > artifact.txt
  artifacts:
    paths: [artifact.txt]

child-pipeline:
  stage: test
  trigger:
    include: child.yml
    strategy: mirror
  variables:
    PARENT_PIPELINE_ID: $CI_PIPELINE_ID

```

- Child pipeline (`child.yml`):

yaml


```

use-artifact:
  script: cat artifact.txt
  needs:
    - pipeline: $PARENT_PIPELINE_ID
      job: create-artifact

```

In this example, the `create-artifact` job in the parent pipeline creates some artifacts. The `child-pipeline` job triggers a child pipeline, and passes the `CI_PIPELINE_ID` variable to the child pipeline as a new `PARENT_PIPELINE_ID` variable. The child pipeline can use that variable in `needs: pipeline` to download artifacts from the parent pipeline. Having the `create-artifact` and `child-pipeline` jobs in subsequent stages ensures that the `use-artifact` job only executes when `create-artifact` has successfully finished.

Additional details:

- The `pipeline` attribute does not accept the current pipeline ID (`$CI_PIPELINE_ID`). To download artifacts from a job in the current pipeline, use `needs: artifacts`.
- You cannot use `needs: pipeline: job` in a [trigger job](#), or to fetch artifacts from a [multi-project pipeline](#). To fetch artifacts from a multi-project pipeline use `needs: project`.
- The job listed in `needs: pipeline: job` must complete with a status of `success` or the artifacts can't be fetched. [Issue 367229](#) proposes to allow fetching artifacts from any job with artifacts.

`needs: optional`

To need a job that sometimes does not exist in the pipeline, add `optional: true` to the `needs` configuration. If not defined, `optional: false` is the default.

- If the `needs` entry has `optional: true` and the needed job is present in the pipeline, the job waits for it to complete before starting.
- If the needed job is not present, the job can start when all other needs requirements are met.
- If the `needs` section contains only optional jobs, and none are added to the pipeline, the job starts immediately (the same as an empty `needs` entry: `needs: []`).
- If a needed job has `optional: false`, but it was not added to the pipeline, the pipeline fails to start with an error similar to: `'job1' job needs 'job2' job, but it was not added to the pipeline.`

Keyword type: Job keyword. You can use it only as part of a job.

Example of `needs:optional`:

yaml


```

build-job:
  stage: build

test-job1:
  stage: test

test-job2:
  stage: test
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH

deploy-job:
  stage: deploy
  needs:
    - job: test-job2
      optional: true
    - job: test-job1
  environment: production

review-job:
  stage: deploy
  needs:
    - job: test-job2
      optional: true
  environment: review

```

In this example:

- `build-job`, `test-job1`, and `test-job2` start in stage order.
- When the branch is the default branch, `test-job2` is added to the pipeline, so:
 - `deploy-job` waits for both `test-job1` and `test-job2` to complete.
 - `review-job` waits for `test-job2` to complete.
- When the branch is not the default branch, `test-job2` is not added to the pipeline, so:
 - `deploy-job` waits for only `test-job1` to complete, and does not wait for the missing `test-job2`.
 - `review-job` has no other needed jobs and starts immediately (at the same time as `build-job`), like `needs: []`.

Additional details:

- You cannot use `needs:optional` with `needs:parallel:matrix`.

needs:pipeline

You can mirror the pipeline status from an upstream pipeline to a job by using the `needs:pipeline` keyword. The latest pipeline status from the default branch is replicated to the job.

- A full project path, including namespace and group. If the project is in the same group or namespace, you can omit them from the `project` keyword. For example: `project: group/project-name` or `project: project-name`.

Example of `needs: pipeline:`

yaml

```
upstream_status:
  stage: test
  needs:
    pipeline: other/project
```

Additional details:

- If you add the `job` keyword to `needs: pipeline`, the job no longer mirrors the pipeline status. The behavior changes to `needs: pipeline: job`.

`needs: parallel: matrix`

History >

Jobs can use `parallel: matrix` to run a job multiple times in parallel in a single pipeline, but with different variable values for each instance of the job.

Use `needs: parallel: matrix` to execute jobs out-of-order depending on parallelized jobs.

Keyword type: Job keyword. You can use it only as part of a job. Must be used with `needs: job`.

Supported values: An array of hashes of matrix identifiers:

- The identifiers and values must be selected from the identifiers and values defined in the `parallel: matrix` job.
- You can use [matrix expressions](#).

Example of `needs: parallel: matrix:`

yaml

```
linux:build:
  stage: build
  script: echo "Building linux..."
  parallel:
    matrix:
      - PROVIDER: aws
        STACK:
          - monitoring
          - app1
          - app2

linux:rspec:
  stage: test
  needs:
    - job: linux:build
      parallel:
        matrix:
          - PROVIDER: aws
            STACK: app1
  script: echo "Running rspec on linux..."
```

The previous example generates the following jobs:

linux:rspec

The linux:rspec job runs as soon as the linux:build: [aws, app1] job finishes.

Additional details:

- You cannot use needs:parallel:matrix with needs:optional.
- The order of the matrix identifiers in needs:parallel:matrix must match the order of the matrix variables in the needed job. For example, reversing the order of the variables in the linux:rspec job in the previous example would be invalid:

yaml

```
linux:rspec:
  stage: test
  needs:
    - job: linux:build
      parallel:
        matrix:
          - STACK: app1          # The variable order does not match `linux:build` and is invalid.
            PROVIDER: aws
  script: echo "Running rspec on linux..."
```

Related topics:

- [Specify a parallelized job using needs with multiple parallelized jobs.](#)
- [Matrix expressions in needs:parallel:matrix.](#)

pages

Use pages to define a [GitLab Pages](#) job that uploads static content to GitLab. The content is then published as a website.

You must:

- Define pages: true to publish a directory named public
- Alternatively, define pages.publish if want to use a different content directory.
- Have a non-empty index.html file in the root of the content directory.

Keyword type: Job keyword or Job name (deprecated). You can use it only as part of a job.

Supported Values:

- A boolean. Uses the default configuration when set to true
- A hash of configuration options, see the following sections for details.

Example of pages:

yaml

```
create-pages:
  stage: deploy
  script:
    - mv my-html-content public
  pages: true # specifies that this is a Pages job and publishes the default public directory
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
  environment: production
```

This example renames the my-html-content/ directory to public/. This directory is exported as an artifact and published with GitLab Pages.

yaml



```
create-pages:
  stage: deploy
  script:
    - echo "nothing to do here"
  pages: # specifies that this is a Pages job and publishes the default public directory
    publish: my-html-content
    expire_in: "1 week"
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
  environment: production
```

This example does not move the directory, but uses the `publish` property directly. It also configures the pages deployment to be unpublished after a week.

Additional details:

- Using `pages` as a job name [is deprecated](#).
- To use `pages` as a job name without triggering a Pages deployment, set the `pages` property to false

pages.publish

History >

Use `pages.publish` to configure the content directory of a [pages job](#).

Keyword type: Job keyword. You can use it only as part of a `pages` job.

Supported values: A path to a directory containing the Pages content. In [GitLab 17.10 and later](#), if not specified, the default `public` directory is used and if specified, this path is automatically appended to [artifacts:paths](#).

Example of `pages.publish`:

yaml



```
create-pages:
  stage: deploy
  script:
    - npx @11ty/eleventy --input=path/to/eleventy/root --output=dist
  pages:
    publish: dist # this path is automatically appended to artifacts:paths
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
  environment: production
```

This example uses [Eleventy](#) to generate a static website and output the generated HTML files into a the `dist/` directory. This directory is exported as an artifact and published with GitLab Pages.

It is also possible to use variables in the `pages.publish` field. For example:

yaml



```
- mkdir -p $CUSTOM_FOLDER/$CUSTOM_PATH
- cp -r public $CUSTOM_FOLDER/$CUSTOM_SUBFOLDER
pages:
  publish: $CUSTOM_FOLDER/$CUSTOM_SUBFOLDER  # this path is automatically appended to artifacts:paths
rules:
  - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
variables:
  CUSTOM_FOLDER: "custom_folder"
  CUSTOM_SUBFOLDER: "custom_subfolder"
```

The publish path specified must be relative to the build root.

Additional details:

- The top-level `publish` keyword [is deprecated](#) and must now be nested under the `pages` keyword

`pages.path_prefix`

Tier: Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

Status: Beta

History >

Use `pages.path_prefix` to configure a path prefix for [parallel deployments](#) of GitLab Pages.

Keyword type: Job keyword. You can use it only as part of a `pages` job.

Supported values:

- A string
- [CI/CD variables](#)
- A combination of both

The given value is converted to lowercase and shortened to 63 bytes. Everything except alphanumeric characters or periods is replaced with a hyphen. Leading and trailing hyphens or periods are not permitted.

Example of `pages.path_prefix`:

`yaml`

```
create-pages:
  stage: deploy
  script:
    - echo "Pages accessible through ${CI_PAGES_URL}"
  pages:  # specifies that this is a Pages job and publishes the default public directory
    path_prefix: "$CI_COMMIT_BRANCH"
```

In this example, a different pages deployment is created for each branch.

`pages.expire_in`

Tier: Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

History >

By default, [parallel deployments](#) expire automatically after 24 hours. To disable this behavior, set the value to `never`.

Keyword type: Job keyword. You can use it only as part of a `pages` job.

Supported values: The expiry time. If no unit is provided, the time is in seconds. Variables are also supported. Valid values include:

- `'42'`
- `42 seconds`
- `3 mins 4 sec`
- `2 hrs 20 min`
- `2h20min`
- `6 mos 1 day`
- `47 yrs 6 mos and 4d`
- `3 weeks and 2 days`
- `never`
- `$DURATION`

Example of `pages.expire_in`:

yaml

```
create-pages:
  stage: deploy
  script:
    - echo "Pages accessible through ${CI_PAGES_URL}"
  pages: # specifies that this is a Pages job and publishes the default public directory
  expire_in: 1 week
```

parallel

History >

Use `parallel` to run a job multiple times in parallel in a single pipeline.

Multiple runners must exist, or a single runner must be configured to run multiple jobs concurrently.

Parallel jobs are named sequentially from `job_name 1/N` to `job_name N/N`.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- A numeric value from `1` to `200`.

Example of `parallel`:

yaml

```
test:
  script: rspec
  parallel: 5
```

This example creates 5 jobs that run in parallel, named `test 1/5` to `test 5/5`.

Additional details:

- Create more jobs running in parallel than available runners. Excess jobs are queued and marked `pending` while waiting for an available runner.
- Fail with a `job_activity_limit_exceeded` error if creating the pipeline would cause the total number of jobs across all active pipelines to [exceed the instance limit](#).

Related topics:

- [Parallelize large jobs](#).

`parallel:matrix`

History >

Use `parallel:matrix` to run a job multiple times in parallel in a single pipeline, but with different variable values for each instance of the job.

Multiple runners must exist, or a single runner must be configured to run multiple jobs concurrently.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: An array of hashes of variables:

- The matrix identifiers, which become the variable names, can use only numbers, letters, and underscores (`_`).
- The values must be either a string, or an array of strings.
- The number of permutations cannot exceed 200.

Example of `parallel:matrix`:

yaml

```
deploystacks:
  stage: deploy
  script:
    - bin/deploy
  parallel:
    matrix:
      - PROVIDER: aws
        STACK:
          - monitoring
          - app1
          - app2
      - PROVIDER: [gcp, vultr]
        STACK: [data, processing]
  environment: $PROVIDER/$STACK
```

📄

The example generates 7 parallel `deploystacks` jobs, each with different values for `PROVIDER` and `STACK`:

- `deploystacks: [aws, monitoring]`
- `deploystacks: [aws, app1]`
- `deploystacks: [aws, app2]`
- `deploystacks: [gcp, data]`
- `deploystacks: [gcp, processing]`
- `deploystacks: [vultr, data]`
- `deploystacks: [vultr, processing]`

Additional details:

- You cannot use the matrix values as variables for `rules:if`.
- You cannot create multiple matrix configurations with the same values but different names. Job names are generated from the matrix values, not the names, so matrix entries with identical values generate identical job names that overwrite each other. For example, this `test` configuration would try to create two series of identical jobs, but the `OS2` versions overwrite the `OS` versions:

yaml

```

test:
  parallel:
    matrix:
      - OS: [ubuntu]
        PROVIDER: [aws, gcp]
      - OS2: [ubuntu]
        PROVIDER: [aws, gcp]

```

Related topics:

- [Run a one-dimensional matrix of parallel jobs.](#)
- [Run a matrix of triggered parallel jobs.](#)
- [Select different runner tags for each parallel matrix job.](#)
- [Matrix expressions in `needs:parallel:matrix`.](#)

release

Use `release` to create a [release](#).

The release job must have access to the `glab CLI`, which must be in the `$PATH`.

If you use the [Docker executor](#), you can use this image from the GitLab container registry: `registry.gitlab.com/gitlab-org/cli:latest`

If you use the [Shell executor](#) or similar, [install `glab CLI`](#) on the server where the runner is registered.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: The `release` subkeys:

- `tag_name`
- `tag_message` (optional)
- `name` (optional)
- `description`
- `ref` (optional)
- `milestones` (optional)
- `released_at` (optional)
- `assets:links` (optional)

Example of `release` keyword:

yaml


```
rules:
  - if: $CI_COMMIT_TAG          # Run this job when a tag is created manually
script:
  - echo "Running the release job."
release:
  tag_name: $CI_COMMIT_TAG
  name: 'Release $CI_COMMIT_TAG'
  description: 'Release created using the CLI.'
```

This example creates a release:

- When you push a Git tag.
- When you add a Git tag in the UI at **Code > Tags**.

Additional details:

- Release jobs must include the `script` keyword. A release job can use the output from script commands. If you don't need the script, you can use a placeholder:

yaml

```
script:
  - echo "release job"
```

For more details, see [issue 223856](#), which aims to remove this restriction.

- The `release` section executes after the `script` keyword and before the `after_script`.
- A release is created only if the job's main script succeeds.
- If the release already exists, it is not updated and the job with the `release` keyword fails.

Related topics:

- [CI/CD example of the `release` keyword](#).
- [Create multiple releases in a single pipeline](#).
- [Use a custom SSL CA certificate authority](#).

`release:tag_name`

Required. The Git tag for the release.

If the tag does not exist in the project yet, it is created at the same time as the release. New tags use the SHA associated with the pipeline.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- A tag name.

CI/CD variables [are supported](#).

Example of `release:tag_name`:

To create a release when a new tag is added to the project:

- Use the `$CI_COMMIT_TAG` CI/CD variable as the `tag_name`.
- Use `rules:if` to configure the job to run only for new tags.

yaml


```
tag_name: $CI_COMMIT_TAG
description: 'Release description'
rules:
  - if: $CI_COMMIT_TAG
```

To create a release and a new tag at the same time, your `rules` should **not** configure the job to run only for new tags. A semantic versioning example:

yaml

```
job:
  script: echo "Running the release job and creating a new tag."
  release:
    tag_name: ${MAJOR}_${MINOR}_${REVISION}
    description: 'Release description'
  rules:
    - if: $CI_PIPELINE_SOURCE == "schedule"
```

release:tag_message

If the tag does not exist, the newly created tag is annotated with the message specified by `tag_message`. If omitted, a lightweight tag is created.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- A text string.

Example of release:tag_message:

yaml

```
release_job:
  stage: release
  release:
    tag_name: $CI_COMMIT_TAG
    description: 'Release description'
    tag_message: 'Annotated tag message'
```

release:name

The release name. If omitted, it is populated with the value of `release: tag_name`.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- A text string.

Example of release:name:

yaml

```
release_job:
  stage: release
  release:
    name: 'Release $CI_COMMIT_TAG'
```

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- A string with the long description.
- The path to a file that contains the description.
 - The file location must be relative to the project directory (`$CI_PROJECT_DIR`).
 - If the file is a symbolic link, it must be in the `$CI_PROJECT_DIR`.
 - The `./path/to/file` and filename can't contain spaces.

Example of `release:description:`

yaml

```

job:
  release:
    tag_name: ${MAJOR}_${MINOR}_${REVISION}
    description: './path/to/CHANGELOG.md'
```



Additional details:

- The `description` is evaluated by the shell that runs `glab`. You can use CI/CD variables to define the description, but some shells [use different syntax](#) to reference variables. Similarly, some shells might require special characters to be escaped. For example, backticks (```) might need to be escaped with a backslash (`\`).

`release:ref`

The `ref` for the release, if the `release: tag_name` doesn't exist yet.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- A commit SHA, another tag name, or a branch name.

`release:milestones`

The title of each milestone the release is associated with.

`release:released_at`

The date and time when the release is ready.

Supported values:

- A date enclosed in quotes and expressed in ISO 8601 format.

Example of `release:released_at:`

yaml

```

released_at: '2021-03-15T08:00:00Z'
```



Additional details:

- If it is not defined, the current date and time is used.

Example of `release:assets:links:`

yaml

```
assets:
  links:
    - name: 'asset1'
      url: 'https://example.com/assets/1'
    - name: 'asset2'
      url: 'https://example.com/assets/2'
      filepath: '/pretty/url/1' # optional
      link_type: 'other' # optional
```

resource_group

Use `resource_group` to create a [resource group](#) that ensures a job is mutually exclusive across different pipelines for the same project.

For example, if multiple jobs that belong to the same resource group are queued simultaneously, only one of the jobs starts. The other jobs wait until the `resource_group` is free.

Resource groups behave similar to semaphores in other programming languages.

You can choose a [process mode](#) to strategically control the job concurrency for your deployment preferences. The default process mode is `unordered`. To change the process mode of a resource group, use the [API](#) to send a request to edit an existing resource group.

You can define multiple resource groups per environment. For example, when deploying to physical devices, you might have multiple physical devices. Each device can be deployed to, but only one deployment can occur per device at any given time.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- Only letters, digits, `-`, `_`, `/`, `$`, `{`, `}`, `.`, and spaces. It can't start or end with `/`. CI/CD variables [are supported](#).

Example of `resource_group:`

yaml

```
deploy-to-production:
  script: deploy
  resource_group: production
```

In this example, two `deploy-to-production` jobs in two separate pipelines can never run at the same time. As a result, you can ensure that concurrent deployments never happen to the production environment.

Related topics:

- [Pipeline-level concurrency control with cross-project/parent-child pipelines.](#)

retry

Use `retry` to configure how many times a job is retried if it fails. If not defined, defaults to `0` and jobs do not retry.

When a job fails, the job is processed up to two more times, until it succeeds or reaches the maximum number of retries.

By default, all failure types cause the job to be retried. Use `retry:when` or `retry:exit_codes` to select which failures to retry on.

- 0 (default), 1, or 2.

Example of `retry`:

yaml

```
test:
  script: rspec
  retry: 2

test_advanced:
  script:
    - echo "Run a script that results in exit code 137."
    - exit 137
  retry:
    max: 2
    when: runner_system_failure
    exit_codes: 137
```

`test_advanced` will be retried up to 2 times if the exit code is 137 or if it had a runner system failure.

`retry:when`

Use `retry:when` with `retry:max` to retry jobs for only specific failure cases. `retry:max` is the maximum number of retries, like `retry`, and can be 0, 1, or 2.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

- A single failure type, or an array of one or more failure types:
- `always`: Retry on any failure (default).
- `unknown_failure`: Retry when the failure reason is unknown.
- `script_failure`: Retry when:
 - The script failed.
 - The runner failed to pull the Docker image. For `docker`, `docker+machine`, `kubernetes` [executors](#).
- `api_failure`: Retry on API failure.
- `stuck_or_timeout_failure`: Retry when the job got stuck or timed out.
- `runner_system_failure`: Retry if there is a runner system failure (for example, job setup failed).
- `runner_unsupported`: Retry if the runner is unsupported.
- `stale_schedule`: Retry if a delayed job could not be executed.
- `job_execution_timeout`: Retry if the script exceeded the maximum execution time set for the job.
- `archived_failure`: Retry if the job is archived and can't be run.
- `unmet_prerequisites`: Retry if the job failed to complete prerequisite tasks.
- `scheduler_failure`: Retry if the scheduler failed to assign the job to a runner.
- `data_integrity_failure`: Retry if there is an unknown job problem.

Example of `retry:when` (single failure type):

yaml

```
max: 2
when: runner_system_failure
```

If there is a failure other than a runner system failure, the job is not retried.

Example of `retry:when` (array of failure types):

yaml

```
test:
  script: rspec
  retry:
    max: 2
    when:
      - runner_system_failure
      - stuck_or_timeout_failure
```

retry:exit_codes

History >

Use `retry:exit_codes` with `retry:max` to retry jobs for only specific failure cases. `retry:max` is the maximum number of retries, like `retry`, and can be 0, 1, or 2.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

- A single exit code.
- An array of exit codes.

Example of `retry:exit_codes`:

yaml

```
test_job_1:
  script:
    - echo "Run a script that results in exit code 1. This job isn't retried."
    - exit 1
  retry:
    max: 2
    exit_codes: 137

test_job_2:
  script:
    - echo "Run a script that results in exit code 137. This job will be retried."
    - exit 137
  retry:
    max: 1
    exit_codes:
      - 255
      - 137
```

Related topics:

You can specify the number of [retry attempts for certain stages of job execution](#) using variables.

Rules are evaluated when the pipeline is created, and evaluated in order. When a match is found, no more rules are checked and the job is either included or excluded from the pipeline depending on the configuration. If no rules match, the job is not added to the pipeline.

`rules` accepts an array of rules. Each rules must have at least one of:

- `if`
- `changes`
- `exists`
- `when`

Rules can also optionally be combined with:

- `allow_failure`
- `needs`
- `variables`
- `interruptible`

You can combine multiple keywords together for [complex rules](#).

The job is added to the pipeline:

- If an `if`, `changes`, or `exists` rule matches, and is configured with `when: on_success` (default if not defined), `when: delayed`, or `when: always`.
- If a rule is reached that is only `when: on_success`, `when: delayed`, or `when: always`.

The job is not added to the pipeline:

- If no rules match.
- If a rule matches and has `when: never`.

For additional examples, see [Specify when jobs run with rules](#).

`rules:if`

Use `rules:if` clauses to specify when to add a job to a pipeline:

- If an `if` statement is true, add the job to the pipeline.
- If an `if` statement is true, but it's combined with `when: never`, do not add the job to the pipeline.
- If an `if` statement is false, check the next `rules` item (if any more exist).

`if` clauses are evaluated:

- Based on the values of [CI/CD variables](#) or [predefined CI/CD variables](#), with [some exceptions](#).
- In order, following [rules execution flow](#).

Keyword type: Job-specific and pipeline-specific. You can use it as part of a job to configure the job behavior, or with [workflow](#) to configure the pipeline behavior.

Supported values:

- A [CI/CD variable expression](#).

Example of `rules:if`:

yaml




```
- if: $CI_MERGE_REQUEST_SOURCE_BRANCH_NAME =~ /^feature/ && $CI_MERGE_REQUEST_TARGET_BRANCH_NAME !=
$CI_DEFAULT_BRANCH
  when: never
- if: $CI_MERGE_REQUEST_SOURCE_BRANCH_NAME =~ /^feature/
  when: manual
  allow_failure: true
- if: $CI_MERGE_REQUEST_SOURCE_BRANCH_NAME
```

Additional details:

- You cannot use [nested variables](#) with `if`. See [issue 327780](#) for more details.
- If a rule matches and has no `when` defined, the rule uses the `when` defined for the job, which defaults to `on_success` if not defined.
- You can mix `when` at the job-level with `when` in rules. `when` configuration in `rules` takes precedence over `when` at the job-level.
- Unlike variables in `script` sections, variables in rules expressions are always formatted as `$VARIABLE`.
 - You can use `rules:if` with `include` to [conditionally include other configuration files](#).
- CI/CD variables on the right side of `=~` and `!~` expressions are [evaluated as regular expressions](#).

Related topics:

- [Common `if` expressions for `rules`](#).
- [Avoid duplicate pipelines](#).
- [Use `rules` to run merge request pipelines](#).

`rules:changes`

Use `rules:changes` to specify when to add a job to a pipeline by checking for changes to specific files.

For new branch pipelines or when there is no Git `push` event, `rules: changes` always evaluates to true and the job always runs. Pipelines like tag pipelines, scheduled pipelines, and manual pipelines, all do **not** have a Git `push` event associated with them. To cover these cases, use `rules: changes: compare_to` to specify the branch to compare against the pipeline ref.

If you do not use `compare_to`, you should use `rules: changes` only with [branch pipelines](#) or [merge request pipelines](#), though `rules: changes` still evaluates to true when creating a new branch. With:

- Merge request pipelines, `rules:changes` compares the changes with the target MR branch.
- Branch pipelines, `rules:changes` compares the changes with the previous commit on the branch.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

An array including any number of:

- Paths to files. The file paths can include [CI/CD variables](#).
- Wildcard paths for:
 - Single directories, for example `path/to/directory/*`.
 - A directory and all its subdirectories, for example `path/to/directory/**/*`.
- Wildcard [glob](#) [↗](#) paths for all files with the same extension or multiple extensions, for example `*.md` or `path/to/directory/*.{rb,py,sh}`.
- Wildcard paths to files in the root directory, or all directories, wrapped in double quotes. For example `"*.json"` or `"**/*.json"`.

Example of `rules:changes`:


```
script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    changes:
      - Dockerfile
    when: manual
    allow_failure: true

docker build alternative:
variables:
  DOCKERFILES_DIR: 'path/to/dockerfiles'
script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
rules:
  - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    changes:
      - $DOCKERFILES_DIR/**/*
```

In this example:

- If the pipeline is a merge request pipeline, check `Dockerfile` and the files in `$DOCKERFILES_DIR/**/*` for changes.
- If `Dockerfile` has changed, add the job to the pipeline as a manual job, and the pipeline continues running even if the job is not triggered (`allow_failure: true`).
- If a file in `$DOCKERFILES_DIR/**/*` has changed, add the job to the pipeline.
- If no listed files have changed, do not add either job to any pipeline (same as `when: never`).

Additional details:

- Glob patterns are interpreted with Ruby's `File.fnmatch` with the `flags` `File::FNM_PATHNAME | File::FNM_DOTMATCH | File::FNM_EXTGLOB`.
- For performance reasons, GitLab performs a maximum of 50,000 checks against `changes` patterns or file paths. After the 50,000th check, rules with patterned globs always match. In other words, the `changes` rule always assumes a match when more than 50,000 files changed, or if there are fewer than 50,000 changed files but the `changes` rules are checked more than 50,000 times.
- A maximum of 50 patterns or file paths can be defined per `rules:changes` section.
- `changes` resolves to `true` if any of the matching files are changed (an `OR` operation).
- For additional examples, see [Specify when jobs run with rules](#).
- You can use the `$` character for both variables and paths. For example, if the `$VAR` variable exists, its value is used. If it does not exist, the `$` is interpreted as being part of a path.
- Do not use `./`, double slashes (`//`), or any other kind of relative path. Paths are matched with exact string comparison, they are not evaluated like in a shell.

Related topics:

- [Jobs or pipelines can run unexpectedly when using rules: changes](#).

`rules:changes:paths`

History >

Use `rules:changes` to specify that a job only be added to a pipeline when specific files are changed, and use `rules:changes:paths` to specify the files.

`rules:changes:paths` is the same as using `rules:changes` without any subkeys. All additional details and related topics are the same.

Keyword type: Job keyword. You can use it only as part of a job.

Example of `rules:changes:paths:`

yaml

```

docker-build-1:
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      changes:
        - Dockerfile

docker-build-2:
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      changes:
        paths:
          - Dockerfile

```

In this example, both jobs have the same behavior.

`rules:changes:compare_to`

History >

Use `rules:changes:compare_to` to specify which ref to compare against for changes to the files listed under `rules:changes:paths`.

Keyword type: Job keyword. You can use it only as part of a job, and it must be combined with `rules:changes:paths`.

Supported values:

- A branch name, like `main`, `branch1`, or `refs/heads/branch1`.
- A tag name, like `tag1` or `refs/tags/tag1`.
- A commit SHA, like `2fg31ga14b`.

CI/CD variables [are supported](#).

Example of `rules:changes:compare_to:`

yaml

```

docker build:
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      changes:
        paths:
          - Dockerfile
      compare_to: 'refs/heads/branch1'

```

In this example, the `docker build` job is only included when the `Dockerfile` has changed relative to `refs/heads/branch1` and the pipeline source is a merge request event.

Additional details:

- Using `compare_to` in some situation can cause unexpected results:
 - With [merged results pipelines](#), because the comparison base is an internal commit that GitLab creates.
 - In a forked project, see [issue 424584](#).

rules:exists

History >

Use `exists` to run a job when certain files or directories exist in the repository.

Keyword type: Job keyword. You can use it as part of a job or an `include`.

Supported values:

- An array of file or directory paths. Paths are relative to the project directory (`$CI_PROJECT_DIR`) and can't directly link outside it. File paths can use glob patterns and [CI/CD variables](#).

Example of `rules:exists`:

yaml

```
job1:
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
  rules:
    - exists:
      - Dockerfile

job2:
  variables:
    DOCKERPATH: "**/Dockerfile"
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
  rules:
    - exists:
      - $DOCKERPATH
```

In this example:

- `job1` runs if a `Dockerfile` exists in the root directory of the repository.
- `job2` runs if a `Dockerfile` exists anywhere in the repository.

Additional details:

- Glob patterns are interpreted with Ruby's `File.fnmatch` with the `flags` `File::FNM_PATHNAME | File::FNM_DOTMATCH | File::FNM_EXTGLOB`.
- For performance reasons, GitLab performs a maximum of 50,000 checks against `exists` patterns or file paths. After the 50,000th check, rules with patterned globs always match. In other words, the `exists` rule always assumes a match in projects with more than 50,000 files, or if there are fewer than 50,000 files but the `exists` rules are checked more than 50,000 times.
 - If there are multiple patterned globs, the limit is 50,000 divided by the number of globs. For example, a rule with 5 patterned globs has file limit of 10,000.
- A maximum of 50 patterns or file paths can be defined per `rules:exists` section.
- `exists` resolves to `true` if any of the listed files are found (an `OR` operation).
- With job-level `rules:exists`, GitLab searches for the files in the project and ref that runs the pipeline. When using `include` with `rules:exists`, GitLab searches for the files or directories in the project and ref of the file that contains the `include` section. The project containing the `include` section can be different than the project running the pipeline when using:
 - [Nested includes](#).
 - [Compliance pipelines](#).
- `rules:exists` cannot search for the presence of [artifacts](#), because `rules` evaluation happens before jobs run and artifacts are fetched.
- To test the existence of a directory, the path must end with a forward slash (/)

`rules:exists:paths` is the same as using `rules:exists` without any subkeys. All additional details are the same.

Keyword type: Job keyword. You can use it as part of a job or an `include`.

Supported values:

- An array of file paths.

Example of `rules:exists:paths`:

yaml

```
docker-build-1:
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      exists:
        - Dockerfile

docker-build-2:
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      exists:
        paths:
          - Dockerfile
```

In this example, both jobs have the same behavior.

rules:exists:project

History >

Use `rules:exists:project` to specify the location in which to search for the files listed under `rules:exists:paths`. Must be used with `rules:exists:paths`.

Keyword type: Job keyword. You can use it as part of a job or an `include`, and it must be combined with `rules:exists:paths`.

Supported values:

- `exists:project`: A full project path, including namespace and group.
- `exists:ref`: Optional. The commit ref to use to search for the file. The ref can be a tag, branch name, or SHA. Defaults to the `HEAD` of the project when not specified.

Example of `rules:exists:project`:

yaml

```
docker build:
  script: docker build -t my-image:$CI_COMMIT_REF_SLUG .
  rules:
    - exists:
        paths:
          - Dockerfile
        project: my-group/my-project
        ref: v1.0.0
```

In this example, the `docker build` job is only included when the `Dockerfile` exists in the project `my-group/my-project` on the commit tagged with `v1.0.0`.

`when`, but with slightly different input options.

If a `rules:when` rule is not combined with `if`, `changes`, or `exists`, it always matches if reached when evaluating a job’s rules.

Keyword type: Job-specific. You can use it only as part of a job.

Supported values:

- `on_success` (default): Run the job only when no jobs in earlier stages fail.
- `on_failure`: Run the job only when at least one job in an earlier stage fails.
- `never`: Don’t run the job regardless of the status of jobs in earlier stages.
- `always`: Run the job regardless of the status of jobs in earlier stages.
- `manual`: Add the job to the pipeline as a [manual job](#). The default value for `allow_failure` changes to `false`.
- `delayed`: Add the job to the pipeline as a [delayed job](#).

Example of `rules:when`:

yaml

```
job1:
  rules:
    - if: $CI_COMMIT_REF_NAME == $CI_DEFAULT_BRANCH
    - if: $CI_COMMIT_REF_NAME =~ /feature/
      when: delayed
    - when: manual
  script:
    - echo
```

In this example, `job1` is added to pipelines:

- For the default branch, with `when: on_success` which is the default behavior when `when` is not defined.
- For feature branches as a delayed job.
- In all other cases as a manual job.

Additional details:

- When evaluating the status of jobs for `on_success` and `on_failure`:
 - Jobs with `allow_failure: true` in earlier stages are considered successful, even if they failed.
 - Skipped jobs in earlier stages, for example [manual jobs that have not been started](#), are considered successful.
- When using `rules:when: manual` to [add a manual job](#):
 - `allow_failure` becomes `false` by default. This default is the opposite of using `when: manual` to add a manual job.
 - To achieve the same behavior as `when: manual` defined outside of `rules`, set `rules: allow_failure` to `true`.

`rules:allow_failure`

Use `allow_failure: true` in `rules` to allow a job to fail without stopping the pipeline.

You can also use `allow_failure: true` with a manual job. The pipeline continues running without waiting for the result of the manual job. `allow_failure: false` combined with `when: manual` in rules causes the pipeline to wait for the manual job to run before continuing.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `true` or `false`. Defaults to `false` if not defined.

yaml



```
job:
  script: echo "Hello, Rules!"
  rules:
    - if: $CI_MERGE_REQUEST_TARGET_BRANCH_NAME == $CI_DEFAULT_BRANCH
      when: manual
      allow_failure: true
```

If the rule matches, then the job is a manual job with `allow_failure: true`.

Additional details:

- The rule-level `rules:allow_failure` overrides the job-level `allow_failure`, and only applies when the specific rule triggers the job.

rules:needs

History >

Use `needs` in rules to update a job's `needs` for specific conditions. When a condition matches a rule, the job's `needs` configuration is completely replaced with the `needs` in the rule.

Keyword type: Job-specific. You can use it only as part of a job.

Supported values:

- An array of job names as strings.
- A hash with a job name, optionally with additional attributes.
- An empty array (`[]`), to set the job needs to none when the specific condition is met.

Example of `rules:needs`:

yaml



```
build-dev:
  stage: build
  rules:
    - if: $CI_COMMIT_BRANCH != $CI_DEFAULT_BRANCH
      script: echo "Feature branch, so building dev version..."

build-prod:
  stage: build
  rules:
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      script: echo "Default branch, so building prod version..."

tests:
  stage: test
  rules:
    - if: $CI_COMMIT_BRANCH != $CI_DEFAULT_BRANCH
      needs: ['build-dev']
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
      needs: ['build-prod']
  script: echo "Running dev specs by default, or prod specs when default branch..."
```

In this example:

- If the pipeline runs on a branch that is not the default branch, and therefore the rule matches the first condition, the `specs` job needs the `build-dev` job.

Additional details:

- `needs` in rules override any `needs` defined at the job-level. When overridden, the behavior is same as [job-level needs](#).
- `needs` in rules can accept [artifacts](#) and [optional](#).

`rules:variables`

Use [variables](#) in `rules` to define variables for specific conditions.

Keyword type: Job-specific. You can use it only as part of a job.

Supported values:

- A hash of variables in the format `VARIABLE-NAME: value`.

Example of `rules:variables`:

yaml

```
job:
  variables:
    DEPLOY_VARIABLE: "default-deploy"
  rules:
    - if: $CI_COMMIT_REF_NAME == $CI_DEFAULT_BRANCH
      variables:
        DEPLOY_VARIABLE: "deploy-production" # Override DEPLOY_VARIABLE defined
        DEPLOY_VARIABLE: "deploy-production" # at the job level.
    - if: $CI_COMMIT_REF_NAME =~ /feature/
      variables:
        IS_A_FEATURE: "true" # Define a new variable.
  script:
    - echo "Run script with $DEPLOY_VARIABLE as an argument"
    - echo "Run another script if $IS_A_FEATURE exists"
```

`rules:interruptible`

History >

Use `interruptible` in rules to update a job's [interruptible](#) value for specific conditions.

Keyword type: Job-specific. You can use it only as part of a job.

Supported values:

- `true` or `false`.

Example of `rules:interruptible`:

yaml

```
job:
  script: echo "Hello, Rules!"
  interruptible: true
  rules:
    - if: $CI_COMMIT_REF_NAME == $CI_DEFAULT_BRANCH
      interruptible: false # Override interruptible defined at the job level.
    - when: on_success
```

Additional details:

run

Status: Experiment

History >

 This feature is available for testing, but not ready for production use.

Use `run` to define a series of [steps](#) to be executed in a job. Each step can be either a script or a predefined step.

You can also provide optional environment variables and inputs.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- An array of hashes, where each hash represents a step with the following possible keys:
 - `name`: A string representing the name of the step.
 - `script`: A string containing shell commands to execute.
 - `step`: A string identifying a predefined step to run.
 - `env`: Optional. A hash of environment variables specific to this step.
 - `inputs`: Optional. A hash of input parameters for predefined steps.

Each array entry must have a `name`, and one `script` or `step` (but not both).

Example of `run`:

yaml

```
job:
  run:
    - name: 'hello_steps'
      script: 'echo "hello from step1"'
    - name: 'bye_steps'
      step: gitlab.com/gitlab-org/ci-cd/runner-tools/echo-step@main
      inputs:
        echo: 'bye steps!'
      env:
        var1: 'value 1'
```



In this example, the job has two steps:

- `hello_steps` runs the `echo` shell command.
- `bye_steps` uses a predefined step with an environment variable and an input parameter.

Additional details:

- A step can have either a `script` or a `step` key, but not both.
- A `run` configuration cannot be used together with existing `script`, `after_script` or `before_script` keywords.
- Multi-line scripts can be defined using [YAML block scalar syntax](#).

script

Use `script` to specify commands for the runner to execute.

Supported values: An array including:

- Single line commands.
- Long commands [split over multiple lines](#).
- [YAML anchors](#).

CI/CD variables [are supported](#).

Example of `script`:

`yaml`

```
job1:
  script: "bundle exec rspec"

job2:
  script:
    - uname -a
    - bundle exec rspec
```



Additional details:

- When you use [these special characters in](#) `script`, you must use single quotes (`'`) or double quotes (`"`).

Related topics:

- You can [ignore non-zero exit codes](#).
- [Use color codes with](#) `script` to make job logs easier to review.
- [Create custom collapsible sections](#) to simplify job log output.

secrets

Tier: Premium, Ultimate

Offering: GitLab.com, GitLab Self-Managed, GitLab Dedicated

Use `secrets` to specify [CI/CD secrets](#) to:

- Retrieve from an external secrets provider.
- Make available in the job as [CI/CD variables](#) (`file` type by default).

secrets:vault

History >

Use `secrets:vault` to specify secrets provided by a [HashiCorp Vault](#) .

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `engine:name`: Name of the secrets engine. Can be one of `kv-v2` (default), `kv-v1`, or `generic`.
- `engine:path`: Path to the secrets engine.
- `path`: Path to the secret.
- `field`: Name of the field where the password is stored.

yaml



```
job:
  secrets:
    DATABASE_PASSWORD: # Store the path to the secret in this CI/CD variable
    vault: # Translates to secret: `ops/data/production/db`, field: `password`
    engine:
      name: kv-v2
      path: ops
    path: production/db
    field: password
```

You can shorten this syntax. With the short syntax, `engine:name` and `engine:path` both default to `kv-v2`:

yaml



```
job:
  secrets:
    DATABASE_PASSWORD: # Store the path to the secret in this CI/CD variable
    vault: production/db/password # Translates to secret: `kv-v2/data/production/db`, field: `password`
```

To specify a custom secrets engine path in the short syntax, add a suffix that starts with `@`:

yaml



```
job:
  secrets:
    DATABASE_PASSWORD: # Store the path to the secret in this CI/CD variable
    vault: production/db/password@ops # Translates to secret: `ops/data/production/db`, field: `password`
```

`secrets:gcp_secret_manager`

History >

Use `secrets:gcp_secret_manager` to specify secrets provided by [GCP Secret Manager](#).

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `name`: Name of the secret.
- `version`: Version of the secret.

Example of `secrets:gcp_secret_manager`:

yaml



```
job:
  secrets:
    DATABASE_PASSWORD:
      gcp_secret_manager:
        name: 'test'
        version: 2
```

Related topics:

- [Use GCP Secret Manager secrets in GitLab CI/CD.](#)

Use `secrets:azure_key_vault` to specify secrets provided by a [Azure Key Vault](#).

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `name`: Name of the secret.
- `version`: Version of the secret.

Example of `secrets:azure_key_vault`:

yaml

```
job:
  secrets:
    DATABASE_PASSWORD:
      azure_key_vault:
        name: 'test'
        version: 'test'
```

Related topics:

- [Use Azure Key Vault secrets in GitLab CI/CD.](#)

secrets:file

Use `secrets:file` to configure the secret to be stored as either a [file](#) or [variable](#) type CI/CD variable

By default, the secret is passed to the job as a `file` type CI/CD variable. The value of the secret is stored in the file and the variable contains the path to the file.

If your software can't use `file` type CI/CD variables, set `file: false` to store the secret value directly in the variable.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- `true` (default) or `false`.

Example of `secrets:file`:

yaml

```
job:
  secrets:
    DATABASE_PASSWORD:
      vault: production/db/password@ops
      file: false
```

Additional details:

- The `file` keyword is a setting for the CI/CD variable and must be nested under the CI/CD variable name, not in the `vault` section.

secrets:token

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- The name of an ID token

Example of `secrets:token:`

yaml

```
job:
  id_tokens:
    AWS_TOKEN:
      aud: https://aws.example.com
    VAULT_TOKEN:
      aud: https://vault.example.com
  secrets:
    DB_PASSWORD:
      vault: gitlab/production/db
      token: $VAULT_TOKEN
```

Additional details:

- When the `token` keyword is not set and there is only one token defined, the defined token will automatically be used.
- If there is more than one token defined, you should specify which token to use by setting the `token` keyword. If you do not specify which token to use, it is not possible to predict which token is used each time the job runs.

services

Use `services` to specify any additional Docker images that your scripts require to run successfully. The `services image` is linked to the image specified in the `image` keyword.

Job configuration and default configuration does not merge together. If the pipeline has `default:services` defined, and the job also has `services`, the job configuration takes precedence and the default configuration is not used.

⚠ To enable inter-service networking, set `FF_NETWORK_PER_BUILD` to `true`. Without this flag, services may not work properly. For more information, see [feature flags](#)

Keyword type: Job keyword. You can use it only as part of a job or in the `default section`.

Supported values: The name of the services image, including the registry path if needed, in one of these formats:

- `<image-name>` (Same as using `<image-name>` with the `latest` tag)
- `<image-name>:<tag>`
- `<image-name>@<digest>`

CI/CD variables [are supported](#), but not for `alias`. To customize `alias` dynamically, use [CI/CD inputs](#) instead.

Example of `services:`

yaml

```
entrypoint: ["/bin/bash"]

services:
  - name: my-postgres:11.7
    alias: db-postgres
    entrypoint: ["/usr/local/bin/db-postgres"]
    command: ["start"]

before_script:
  - bundle install

test:
  script:
    - bundle exec rake spec
```

In this example, GitLab launches two containers for the job:

- A Ruby container that runs the `script` commands.
- A PostgreSQL container. The `script` commands in the Ruby container can connect to the PostgreSQL database at the `db-postgres` hostname.

Additional details:

- Using `services` at the top level, but not in the `default` section, is [deprecated](#).

Related topics:

- [Available settings for `services`](#).
- [Define `services` in the `.gitlab-ci.yml` file](#).
- [Run your CI/CD jobs in Docker containers](#).
- [Use Docker to build Docker images](#).

`services:name`

The full name of the image to use for the service.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values: The name of the service image, including the registry path if needed, in one of these formats:

- `<image-name>` (Same as using `<image-name>` with the `latest` tag)
- `<image-name>:<tag>`
- `<image-name>@<digest>`

CI/CD variables [are supported](#).

Example of `services:name`:

yaml

```
services:
  - name: postgres:11.7
  - name: registry.example.com/my-org/custom-service:latest
```

Additional details:

- Use `alias` to define unique name aliases when using multiple identical service images, or when the service image name is long.
- When used with other service options like `entrypoint`, `command`, or `variables`, the `name` keyword is required.

services:alias

History >

Additional aliases to access the service from the job’s container.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values: A string with one or more aliases separated by spaces or commas.

Example of `services:alias`:

yaml

```
services:
  - name: postgres:11.7
    alias: db,postgres,pg
  - name: mysql:latest
    alias: mysql-1
```

Additional details:

- Multiple aliases can be separated by spaces or commas.
- For more information, see [accessing the services](#). and [using aliases as service container names for the Kubernetes executor](#).

services:docker

History >

Use `services:docker` to pass options to the Docker executor of a GitLab Runner.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

A hash of options for the Docker executor, which can include:

- `platform`: Selects the architecture of the image to pull. When not specified, the default is the same platform as the host runner.
- `user`: Specify the username or UID to use when running the container.

Example of `services:docker`:

yaml

```
arm-sql-job:
  script: echo "Run sql tests in service container"
  image: ruby:2.6
  services:
    - name: super/sql:experimental
      docker:
        platform: arm64/v8
        user: dave
```

Additional details:

- `services:docker:platform` maps to the `docker pull --platform` option [↗](#).
- `services:docker:user` maps to the `docker run --user` option [↗](#).

Use `services:kubernetes` to pass options to the GitLab Runner [Kubernetes executor](#).

Keyword type: Job keyword. You can use it only as part of a job or in the `default` [section](#).

Supported values:

A hash of options for the Kubernetes executor, which can include:

- `user`: Specify the username or UID to use when the container runs. You can also use it to set GID by using the `UID:GID` format.

Example of `services:kubernetes` with only UID:

yaml

```
arm-sql-job:
  script: echo "Run sql tests"
  image: ruby:2.6
  services:
    - name: super/sql:experimental
      kubernetes:
        user: "1001"
```

Example of `services:kubernetes` with both UID and GID:

yaml

```
arm-sql-job:
  script: echo "Run sql tests"
  image: ruby:2.6
  services:
    - name: super/sql:experimental
      kubernetes:
        user: "1001:1001"
```

`services:entrypoint`

A command or script to execute as the container’s entrypoint.

When the Docker container is created, the `entrypoint` is translated to the Docker `--entrypoint` option. The syntax is similar to the [Dockerfile ENTRYPOINT directive](#) [↗](#), where each shell token is a separate string in the array.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` [section](#).

Supported values: An array of strings representing the entrypoint command.

Example of `services:entrypoint`:

yaml

```
services:
  - name: my-postgres:11.7
    entrypoint: ["/usr/local/bin/db-postgres"]
```

`services:command`

Command or script that should be used as the container’s command.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values: An array of strings representing the command.

Example of `services:command:`

yaml

```
services:
  - name: super/sql:latest
    command: ["/usr/bin/super-sql", "run"]
```

`services:variables`

Additional environment variables that are passed exclusively to the service. Service variables are passed exclusively to the service container and are not available to the job container.

The syntax is the same as [job variables](#).

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values: A hash of environment variable names and values.

Example of `services:variables:`

yaml

```
services:
  - name: postgres:11.7
    alias: db
    variables:
      POSTGRES_DB: "my_custom_db"
      POSTGRES_USER: "postgres"
      POSTGRES_PASSWORD: "example"
      PGDATA: "/var/lib/postgresql/data"
```

Additional details:

- Service variables cannot reference themselves, they do not support variable expansion or interpolation.
- Variables defined at the job or pipeline level are automatically passed to services. See [passing CI/CD variables to services](#) for more information.
- Service variables are only available to the specific service they are defined for.

`services:pull_policy`

History >

The pull policy that the runner uses to fetch the Docker image. Requires GitLab Runner 15.1 or later.

Keyword type: Job keyword. You can use it only as part of a job or in the `default` section.

Supported values:

- A single pull policy, or multiple pull policies in an array. Can be `always`, `if-not-present`, or `never`.

Examples of `services:pull_policy:`

yaml

```
- name: postgres:11.6
  pull_policy: if-not-present

job2:
  script: echo "Multiple pull policies."
  services:
    - name: postgres:11.6
      pull_policy: [always, if-not-present]
```

Additional details:

- If the runner does not support the defined pull policy, the job fails with an error similar to: `ERROR: Job failed (system failure): the configured PullPolicies ([always]) are not allowed by AllowedPullPolicies ([never])`.

Related topics:

- [Run your CI/CD jobs in Docker containers.](#)
- [Configure how runners pull images.](#)
- [Set multiple pull policies.](#)

stage

Use `stage` to define which [stage](#) a job runs in. Jobs in the same `stage` can execute in parallel (see **Additional details**).

If `stage` is not defined, the job uses the `test` stage by default.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: A string, which can be a:

- [Default stage](#).
- User-defined stages.

Example of `stage`:

yaml



```
- deploy

job1:
  stage: build
  script:
    - echo "This job compiles code."

job2:
  stage: test
  script:
    - echo "This job tests the compiled code. It runs when the build stage completes."

job3:
  script:
    - echo "This job also runs in the test stage."

job4:
  stage: deploy
  script:
    - echo "This job deploys the code. It runs when the test stage completes."
  environment: production
```

Additional details:

- The stage name must be 255 characters or fewer.
- Jobs can run in parallel if they run on different runners.
- If you have only one runner, jobs can run in parallel if the runner’s [concurrent setting](#) is greater than `1`.

`stage: .pre`

Use the `.pre` stage to make a job run at the start of a pipeline. By default, `.pre` is the first stage in a pipeline. User-defined stages execute after `.pre`. You do not have to define `.pre` in [stages](#).

If a pipeline contains only jobs in the `.pre` or `.post` stages, it does not run. There must be at least one other job in a different stage.

Keyword type: You can only use it with a job’s `stage` keyword.

Example of `stage: .pre`:

yaml

```
stages:
  - build
  - test

job1:
  stage: build
  script:
    - echo "This job runs in the build stage."

first-job:
  stage: .pre
  script:
    - echo "This job runs in the .pre stage, before all other stages."

job2:
  stage: test
  script:
    - echo "This job runs in the test stage."
```

`needs: []` start immediately, ignoring any stage configuration.

- A [pipeline execution policy](#) can define a `.pipeline-policy-pre` stage which runs before `.pre`.

`stage: .post`

Use the `.post` stage to make a job run at the end of a pipeline. By default, `.post` is the last stage in a pipeline. User-defined stages execute before `.post`. You do not have to define `.post` in [stages](#).

If a pipeline contains only jobs in the `.pre` or `.post` stages, it does not run. There must be at least one other job in a different stage.

Keyword type: You can only use it with a job's `stage` keyword.

Example of `stage: .post`:

yaml


```

stages:
  - build
  - test

job1:
  stage: build
  script:
    - echo "This job runs in the build stage."

last-job:
  stage: .post
  script:
    - echo "This job runs in the .post stage, after all other stages."

job2:
  stage: test
  script:
    - echo "This job runs in the test stage."

```

Additional details:

- A [pipeline execution policy](#) can define a `.pipeline-policy-post` stage which runs after `.post`.

`tags`

Use `tags` to select a specific runner from the list of all runners that are available for the project.

When you register a runner, you can specify the runner's tags, for example `ruby`, `postgres`, or `development`. To pick up and run a job, a runner must be assigned every tag listed in the job.

Job configuration and default configuration does not merge together. If the pipeline has `default:tags` defined, and the job also has `tags`, the job configuration takes precedence and the default configuration is not used.

Keyword type: Job keyword. You can use it only as part of a job or in the [default section](#).

Supported values:

- An array of tag names, which are case-sensitive.
- CI/CD variables [are supported](#).

Example of `tags`:

```
tags:
  - ruby
  - postgres
```

In this example, only runners with both the `ruby` and `postgres` tags can run the job.

Additional details:

- The number of tags must be less than `50`.

Related topics:

- [Use tags to control which jobs a runner can run](#)
- [Select different runner tags for each parallel matrix job](#)
- Runner tags for hosted runners:
 - [Hosted runners on Linux](#)
 - [GPU-enabled hosted runners](#)
 - [Hosted runners on macOS](#)
 - [Hosted runners on Windows](#)

`timeout`

Use `timeout` to configure a timeout for a specific job. If the job runs for longer than the timeout, the job fails.

The job-level timeout can be longer than the [project-level timeout](#), but can't be longer than the [runner's timeout](#).

Keyword type: Job keyword. You can use it only as part of a job.

Supported values: A period of time written in natural language. For example, these are all equivalent:

- `3600 seconds`
- `60 minutes`
- `one hour`

Example of `timeout`:

yaml

```
build:
  script: build.sh
  timeout: 3 hours 30 minutes

test:
  script: rspec
  timeout: 3h 30m
```



Additional details:

- The `timeout` keyword is not supported in the `default` configuration. Define `timeout` in individual job configurations instead. For more information, see [issue 213634](#).

`trigger`

- [A child pipeline.](#)

Trigger jobs can use only a limited set of GitLab CI/CD configuration keywords. The keywords available for use in trigger jobs are:

- `allow_failure`.
- `extends`.
- `needs`, but not `needs:project`.
- `only` and `except`.
- `parallel`.
- `rules`.
- `stage`.
- `trigger`.
- `variables`.
- `when` (only with a value of `on_success`, `on_failure`, `always`, or `manual`).
- `resource_group`.
- `environment`.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- For multi-project pipelines, the path to the downstream project. CI/CD variables [are supported](#) in GitLab 15.3 and later, but not [job-only variables](#). Alternatively, use `trigger:project`.
- For child pipelines, use `trigger:include`.

Example of `trigger`:

yaml


```
trigger-multi-project-pipeline:
  trigger: my-group/my-project
```

Additional details:

- You can use `when:manual` in the same job as `trigger`, but you cannot use the API to start `when:manual` trigger jobs. See [issue 284086](#) for more details.
- You cannot [manually specify CI/CD variables](#) before running a manual trigger job.
- [CI/CD variables](#) defined in a top-level `variables` section (globally) or in the trigger job are forwarded to the downstream pipeline as [trigger variables](#).
- [Pipeline variables](#) are not passed to downstream pipelines by default. Use `trigger:forward` to forward these variables to downstream pipelines.
- [Job-only variables](#) are not available in trigger jobs.
- Environment variables [defined in the runner's config.toml](#) are not available to trigger jobs and are not passed to downstream pipelines.
- You cannot use `needs:pipeline:job` in a trigger job.

Related topics:

- [Multi-project pipeline configuration examples.](#)
- To run a pipeline for a specific branch, tag, or commit, you can use a [trigger token](#) to authenticate with the [pipeline triggers API](#). The trigger token is different than the `trigger` keyword.

Use `trigger:inputs` to set the [inputs](#) for a multi-project pipeline when the downstream pipeline configuration uses [spec:inputs](#).

Example of `trigger:inputs`:

yaml

```
trigger:
  - project: 'my-group/my-project'
    inputs:
      website: "My website"
```

`trigger:include`

Use `trigger:include` to declare that a job is a “trigger job” which starts a [child pipeline](#).

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- The path to the child pipeline’s configuration file.

Example of `trigger:include`:

yaml

```
trigger-child-pipeline:
  trigger:
    include: path/to/child-pipeline.gitlab-ci.yml
```

Additional details:

Use:

- `trigger:include:artifact` to trigger a [dynamic child pipeline](#).
- `trigger:include:inputs` to set the [inputs](#) when the downstream pipeline configuration uses [spec:inputs](#).
- `trigger:include:local` for a path to a child pipeline configuration file when:
 - Combining [multiple child pipeline configuration files](#).
 - Combined with `trigger:include:inputs` to pass inputs to the child pipeline. For example:

yaml

```
staging-job:
  trigger:
    include:
      - local: path/to/child-pipeline.yml
    inputs:
      environment: staging
```

- `trigger:include:project` to trigger a child pipeline [with a configuration file in a different project](#). If the file contains additional [include](#) entries, GitLab looks for the files in the project running the pipeline, not the project hosting the file.
- `trigger:include:template` to trigger a child pipeline with a CI/CD template.

Related topics:

- [Child pipeline configuration examples](#).

Use `trigger:include:inputs` to set the [inputs](#) for a child pipeline when the downstream pipeline configuration uses [spec:inputs](#).

Example of `trigger:inputs`:

yaml

```
trigger-job:
  trigger:
    include:
      - local: path/to/child-pipeline.yml
    inputs:
      website: "My website"
```

`trigger:project`

Use `trigger:project` to declare that a job is a “trigger job” which starts a [multi-project pipeline](#).

By default, the multi-project pipeline triggers for the default branch. Use `trigger:branch` to specify a different branch.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- The path to the downstream project. CI/CD variables [are supported](#) in GitLab 15.3 and later, but not [job-only variables](#).

Example of `trigger:project`:

yaml

```
trigger-multi-project-pipeline:
  trigger:
    project: my-group/my-project
```

Example of `trigger:project` for a different branch:

yaml

```
trigger-multi-project-pipeline:
  trigger:
    project: my-group/my-project
    branch: development
```

Related topics:

- [Multi-project pipeline configuration examples](#).
- To run a pipeline for a specific branch, tag, or commit, you can also use a [trigger token](#) to authenticate with the [pipeline triggers API](#). The trigger token is different than the `trigger` keyword.

`trigger:strategy`

Use `trigger:strategy` to force the `trigger` job to wait for the downstream pipeline to complete before it is marked as **success**.

This behavior is different than the default, which is for the `trigger` job to be marked as **success** as soon as the downstream pipeline is created.

- `mirror`: Mirrors the status of the downstream pipeline exactly.
- `depend`: Not recommended, use `mirror` instead. The trigger job status shows **failed**, **success**, or **running**, depending on the downstream pipeline status. See additional details.

Example of `trigger:strategy`:

yaml

```
trigger_job:
  trigger:
    include: path/to/child-pipeline.yml
    strategy: mirror
```

In this example, jobs from subsequent stages wait for the triggered pipeline to successfully complete before starting.

Additional details:

- [Optional manual jobs](#) in the downstream pipeline do not affect the status of the downstream pipeline or the upstream trigger job. The downstream pipeline can complete successfully without running any optional manual jobs.
- By default, jobs in later stages do not start until the trigger job completes.
- [Blocking manual jobs](#) in the downstream pipeline must run before the trigger job is marked as successful or failed.
- When using `strategy:depend` (no longer recommended, use `strategy:mirror` instead):
 - The trigger job shows **running** (🔄) if the downstream pipeline status is **waiting for manual action** (⛔) due to manual jobs.
 - If the downstream pipeline has a failed job, but the job uses `allow_failure: true`, the downstream pipeline is considered successful and the trigger job shows **success**.

`trigger:forward`

History >

Use `trigger:forward` to specify what to forward to the downstream pipeline. You can control what is forwarded to both [parent-child pipelines](#) and [multi-project pipelines](#).

Forwarded variables do not get forwarded again in nested downstream pipelines by default, unless the nested downstream trigger job also uses `trigger:forward`.

Supported values:

- `yaml_variables`: `true` (default), or `false`. When `true`, variables defined in the trigger job are passed to downstream pipelines.
- `pipeline_variables`: `true` or `false` (default). When `true`, [pipeline variables](#) are passed to the downstream pipeline.

Example of `trigger:forward`:

[Run this pipeline manually](#), with the CI/CD variable `MYVAR = my value`:

yaml

```
---

# Default behavior:
---

# - VAR is passed to the child
---

# - MYVAR is not passed to the child
child1:
  trigger:
    include: .child-pipeline.yml
---

# Forward pipeline variables:
---

# - VAR is passed to the child
---

# - MYVAR is passed to the child
child2:
  trigger:
    include: .child-pipeline.yml
  forward:
    pipeline_variables: true
---

# Do not forward YAML variables:
---

# - VAR is not passed to the child
---

# - MYVAR is not passed to the child
child3:
  trigger:
    include: .child-pipeline.yml
  forward:
    yaml_variables: false
```

Additional details:

- CI/CD variables forwarded to downstream pipelines with `trigger:forward` are [pipeline variables](#), which have high precedence. If a variable with the same name is defined in the downstream pipeline, that variable is usually overwritten by the forwarded variable.

when

Use `when` to configure the conditions for when jobs run. If not defined in a job, the default value is `when: on_success`.

Keyword type: Job keyword. You can use it as part of a job. `when: always` and `when: never` can also be used in [workflow:rules](#).

Supported values:

- `on_success` (default): Run the job only when no jobs in earlier stages fail.
- `on_failure`: Run the job only when at least one job in an earlier stage fails.

- `always`: Run the job regardless of the status of jobs in earlier stages.
- `manual`: Add the job to the pipeline as a [manual job](#).
- `delayed`: Add the job to the pipeline as a [delayed job](#).

Example of `when`:

yaml


```

stages:
  - build
  - cleanup_build
  - test
  - deploy
  - cleanup

build_job:
  stage: build
  script:
    - make build

cleanup_build_job:
  stage: cleanup_build
  script:
    - cleanup build when failed
  when: on_failure

test_job:
  stage: test
  script:
    - make test

deploy_job:
  stage: deploy
  script:
    - make deploy
  when: manual
  environment: production

cleanup_job:
  stage: cleanup
  script:
    - cleanup after jobs
  when: always
  
```

In this example, the script:

1. Executes `cleanup_build_job` only when `build_job` fails.
2. Always executes `cleanup_job` as the last step in pipeline regardless of success or failure.
3. Executes `deploy_job` when you run it manually in the GitLab UI.

Additional details:

- When evaluating the status of jobs for `on_success` and `on_failure`:
 - Jobs with `allow_failure: true` in earlier stages are considered successful, even if they failed.
 - Skipped jobs in earlier stages, for example [manual jobs that have not been started](#), are considered successful.
- The default value for `allow_failure` is `true` with `when: manual`. The default value changes to `false` with [rules:when: manual](#).

Related topics:

- `when` can be used with [rules](#) for more dynamic job control.

manual_confirmation

History >

Use `manual_confirmation` with `when: manual` to define a custom confirmation message for manual jobs. If no manual job is defined with `when: manual`, this keyword has no effect.

Manual confirmation works with all manual jobs, including environment stop jobs that use `environment:action: stop`.

Keyword type: Job keyword. You can use it only as part of a job.

Supported values:

- A string with the confirmation message.

Example of `manual_confirmation`:

yaml

```
delete_job:
  stage: post-deployment
  script:
    - make delete
  when: manual
  manual_confirmation: 'Are you sure you want to delete this environment?'

stop_production:
  stage: cleanup
  script:
    - echo "Stopping production environment"
  environment:
    name: production
    action: stop
  when: manual
  manual_confirmation: "Are you sure you want to stop the production environment?"
```

start_in

Use `start_in` to delay the execution of a job for a specified duration after the job is created. You must configure `when: delayed` for the job.

Keyword type: Job keyword. You can use it only as part of a job.

Possible inputs: A period of time in seconds, minutes, or hours. Must be less than or equal to one week. Examples of valid values:

- `'5'` (5 seconds)
- `'10 seconds'`
- `'30 minutes'`
- `'1 hour'`
- `'1 day'`

Example of `start_in`:

yaml


```
- echo "Deploying to production"
when: delayed
start_in: 30 minutes
```

In this example, the `deploy_production` job starts 30 minutes after the previous stage completes.

Additional details:

- The timer starts when the job’s stage begins, not when the previous job finishes.
- To manually start a delayed job immediately, select **Play** (▶) in the pipeline view.
- The minimum delay period is one second and the maximum delay is one week.
- `start_in` only works when `when` is set to `delayed`. If you use any other value for `when`, the configuration is invalid. If a job uses `rules`, `start_in` and `when` must be defined in the `rules`, not at the job level. Otherwise, you receive a validation error: `config key may not be used with 'rules': start_in`.
- `start_in` is not supported with `workflow:rules`, but does not cause any syntax violation.

Related topics:

- [Run a job after a delay](#)

variables

Use `variables` to define [CI/CD variables](#).

Variables can be [defined in a CI/CD job](#), or as a top-level (global) keyword to define [default CI/CD variables](#) for all jobs.

Additional details:

- All YAML-defined variables are also set to any linked [Docker service containers](#).
- YAML-defined variables are meant for non-sensitive project configuration. Store sensitive information in [protected variables](#) or [CI/CD secrets](#).
- [Manual pipeline variables](#) and [scheduled pipeline variables](#) are not passed to downstream pipelines by default. Use `trigger:forward` to forward these variables to downstream pipelines.

Related topics:

- [Predefined variables](#) are variables the runner automatically creates and makes available in the job.
- You can [configure runner behavior with variables](#).

Job variables

You can use job variables in commands in the job’s `script`, `before_script`, or `after_script` sections, and also with some [job keywords](#). Check the **Supported values** section of each job keyword to see if it supports variables.

You cannot use job variables as values for [global keywords](#) like `include`.

Supported values: Variable name and value pairs:

- The name can use only numbers, letters, and underscores (`_`). In some shells, the first character must be a letter.
- The value must be a string.

CI/CD variables [are supported](#).

Example of job variables:

yaml




```
REVIEW_PATH: "/review"
script:
  - deploy-review-script --url $DEPLOY_SITE --path $REVIEW_PATH
```

In this example:

- `review_job` has `DEPLOY_SITE` and `REVIEW_PATH` job variables defined. Both job variables can be used in the `script` section.

Default variables

Variables defined in a top-level `variables` section act as default variables for all jobs.

Each default variable is made available to every job in the pipeline, except when the job already has a variable defined with the same name. The variable defined in the job [takes precedence](#), so the value of the default variable with the same name cannot be used in the job.

Like job variables, you cannot use default variables as values for other global keywords, like `include`.

Supported values: Variable name and value pairs:

- The name can use only numbers, letters, and underscores (`_`). In some shells, the first character must be a letter.
- The value must be a string.

CI/CD variables [are supported](#).

Examples of variables:

yaml



```
variables:
  DEPLOY_SITE: "https://example.com/"

deploy_job:
  stage: deploy
  script:
    - deploy-script --url $DEPLOY_SITE --path "/"
  environment: production

deploy_review_job:
  stage: deploy
  variables:
    DEPLOY_SITE: "https://dev.example.com/"
    REVIEW_PATH: "/review"
  script:
    - deploy-review-script --url $DEPLOY_SITE --path $REVIEW_PATH
  environment: production
```

- [Get support](#)
- [Post on the GitLab forum](#)
- [Privacy statement](#)
- [Use of generative AI](#)
- [Acceptable use of user licenses](#)
- [Cookie Preferences](#)

variables:description

Use the `description` keyword to define a description for a default variable. The description displays with [the prefilled variable name when running a pipeline manually](#).

- A string. You can use Markdown.

Example of `variables:description:`

yaml

```
variables:
  DEPLOY_NOTE:
    description: "The deployment note. Explain the reason for this deployment."
```

Additional details:

- When used without `value`, the variable exists in pipelines that were not triggered manually, and the default value is an empty string (`''`).

`variables:value`

Use the `value` keyword to define a pipeline-level (default) variable’s value. When used with `variables: description`, the variable value is [prefilled when running a pipeline manually](#).

Keyword type: You can only use this keyword with default `variables`, not job `variables`.

Supported values:

- A string.

Example of `variables:value:`

yaml

```
variables:
  DEPLOY_ENVIRONMENT:
    value: "staging"
    description: "The deployment target. Change this variable to 'canary' or 'production' if needed."
```

Additional details:

- If used without `variables: description`, the behavior is the same as `variables`.

`variables:options`

History >

Use `variables:options` to define an array of values that are [selectable in the UI when running a pipeline manually](#).

Must be used with `variables: value`, and the string defined for `value`:

- Must also be one of the strings in the `options` array.
- Is the default selection.

If there is no `description`, this keyword has no effect.

Keyword type: You can only use this keyword with default `variables`, not job `variables`.

Supported values:

- An array of strings.

Example of `variables:options:`

```

DEPLOY_ENVIRONMENT:
  value: "staging"
  options:
    - "production"
    - "staging"
    - "canary"
  description: "The deployment target. Set to 'staging' by default."
```

variables:expand

History >

Use the `expand` keyword to configure a variable to be expandable or not.

Keyword type: You can use this keyword with both default and job `variables`.

Supported values:

- `true` (default): The variable is expandable.
- `false`: The variable is not expandable.

Example of `variables:expand`:

yaml
 

```

variables:
  VAR1: value1
  VAR2: value2 $VAR1
  VAR3:
    value: value3 $VAR1
    expand: false
```

- The result of `VAR2` is `value2 value1`.
- The result of `VAR3` is `value3 $VAR1`.

Additional details:

- The `expand` keyword can only be used with default and job `variables` keywords. You can't use it with `rules:variables` or `workflow:rules:variables`.

Was this page helpful?
 
